

**From Benchmark to Agents:
How SWE-bench Advanced LLM4Code**

A Comparative Literature Review

by

Manyi Wang

Student ID: 225045034

Final Report
in Partial Fulfillment of the
Advanced Operating System Course

School of Data Science
The Chinese University of Hong Kong, Shenzhen

April 2026

Thesis Assessment Committee

Professor CHUNG, Yehching (Instructor)

LIU, Yuxuan (Teaching Assistant)

Abstract

Repository-level repair is the central problem studied in this thesis. This problem asks whether a language model can read a real issue report, inspect a real repository, and produce a patch that survives executable tests. This executable setting is introduced by SWE-bench, which transforms GitHub issues and merged pull requests into benchmark instances for repository-level maintenance. This benchmarked task then becomes the common target for SWE-agent, which gives a language model an agent-computer interface for searching, editing, and testing inside the repository. This agentic response reveals the value of tool feedback and interaction, but this value also raises a control question: how much autonomy is actually necessary once the task has been made precise? This control question motivates Agentless, which replaces an open-ended action loop with explicit stages for localization, repair, reproduction testing, and patch selection.

This sequence is the main argument of the review. SWE-bench defines what counts as success; SWE-agent studies how an LLM should gather evidence and act; Agentless studies how far the same repair process can be decomposed into auditable stages. These roles are different, but the differences are productive because they separate benchmark design, interaction design, and repair control. The separation also clarifies why final resolve rate alone is an incomplete measure of progress. Progress in LLM-based software engineering also depends on context selection, edit reliability, test adequacy, cost, reproducibility, sandboxing, and the ability of human developers to inspect intermediate artifacts.

The thesis compares the three papers across task formulation, repository context selection, interaction design, localization, patch generation, validation, and cost-aware reproducibility. The comparison shows that SWE-bench moves evaluation from short code generation to software maintenance, SWE-agent makes the computer interface a first-class research object, and Agentless turns benchmark-aware repair into a structured generate-and-validate pipeline. The resulting conclusion is hybrid rather than ideological: future systems should use SWE-bench-style benchmarks to define realistic tasks, Agentless-style pipelines as efficient default paths, and SWE-agent-style interaction when uncertainty requires open-ended exploration.

Acknowledgement

I would like to thank Professor Yehching Chung for his wonderful course and thoughtful guidance throughout the semester. I learned a lot, especially in terms of reading research papers, academic writing, and delivering presentations. I would also like to thank our teaching assistant, Mr. Yuxuan Liu, for his help and feedback throughout the semester.

Contents

Abstract	i
Acknowledgement	ii
List of Figures	vi
List of Tables	viii
1 Introduction	1
1.1 Research Questions	2
1.2 Repository-Level Code Repair	2
1.3 Selection of the Three Focal Papers	2
1.4 Contributions of This Review	3
1.5 Methodological Posture	3
1.6 Chapter Outline	4
2 Background and Preliminaries	5
2.1 From Code Completion to Software Engineering	5
2.2 Repository-Level Repair as a Systems Problem	6
2.3 Fault Localization, Context Selection, and Retrieval	7
2.4 Agents, Tools, and Interfaces	7
2.5 Testing, Validation, and Benchmark Validity	8
2.6 Cost, Reproducibility, and Scientific Claims	9
2.7 Review Approach and Broader Literature	9
2.8 Summary	11
3 SWE-bench: Establishing the Common Evaluation Problem	12
3.1 The Problem SWE-bench Defines	12
3.2 Benchmark Construction and Task Representation	13
3.3 Retrieval, Context Length, and Baseline Solvers	14
3.4 Evaluation Strengths and Validity Risks	16
3.5 Relationship to SWE-agent and Agentless	16

3.6	Summary and Transition to SWE-agent	17
4	SWE-agent: A Classic Agent-Based Approach	19
4.1	From Static Prompting to Interactive Repair	19
4.2	Agent-Computer Interface Design	19
4.3	Evaluation on SWE-bench and Related Tasks	21
4.4	Ablations, Behavior, and Failure Modes	22
4.5	Strengths, Limitations, and Scientific Position	23
4.6	Summary and Transition to Agentless	23
5	Agentless: A Classic Pipeline-Based Approach	25
5.1	Why an Agentless Approach Matters	25
5.2	Hierarchical Localization	25
5.3	Patch Generation and Edit Format	27
5.4	Reproduction Tests, Patch Validation, and Ranking	28
5.5	Evaluation, Benchmark Audit, and SWE-bench Lite-S	30
5.6	Strengths, Limitations, and Relationship to Agents	31
5.7	Agentless as a Benchmark-Aware Repair System	32
5.8	Summary	33
6	Comparative Review: Benchmark Definition, Agentic Interaction, and Structured Pipelines	34
6.1	A Shared Framework for the Three Papers	34
6.2	Benchmark Definition and Evaluation Validity	34
6.3	Agent-Based Interaction Versus Pipeline-Based Control	35
6.4	Localization, Context, and Information Flow	36
6.5	Validation, Tests, and Patch Plausibility	36
6.6	Cost, Reproducibility, and Deployment	37
6.7	Toward Hybrid Automated Software Engineering	37
6.8	Summary	40
7	Conclusion and Future Work	41
7.1	Main Findings	41
7.2	Implications for LLM4Code Research	41
7.3	Future Work	42
7.4	Closing Reflection	42
	Bibliography	43
	Appendix	48

A	Supplementary Notes on Benchmarks and Repair Systems	48
A.1	Supplementary Notes on SWE-bench Variants	48
A.2	Supplementary Notes on Model and Tool Assumptions	48
A.3	Closing Note on Supplementary Material	48

List of Figures

3.1	SWE-bench collection pipeline. Real GitHub issues are linked to merged pull requests, installable repository snapshots, and executable tests, turning software maintenance into a benchmarkable task.	13
3.2	Example SWE-bench task instance. The issue text, repository snapshot, reference patch, and tests define a repair problem whose solution must be executable rather than merely plausible.	13
3.3	Distribution of SWE-bench tasks across the twelve source repositories. The spread across projects makes localization and repository-specific compatibility central parts of the task.	14
4.1	SWE-agent overview. The model repeatedly observes repository state, selects interface actions, edits files, and submits the resulting repository state as a patch. 20	
4.2	Agent-computer interface design. SWE-agent treats the textual interface as part of the system, not as a neutral wrapper around the model.	20
4.3	SWE-agent file-viewing and editing interface. Controlled windows and explicit edit feedback reduce the brittleness of raw shell-based editing.	20
4.4	SWE-agent pass@k behavior across repeated runs on SWE-bench Lite. The curve shows that repeated agent trajectories can solve partially different instances. 22	
5.1	Agentless overview. The repair process is decomposed into localization, repair, and patch validation instead of being delegated to an open-ended agent loop. . .	26
5.2	Agentless file skeleton format. The skeleton compresses file content into declarations so the model can localize relevant classes and functions before seeing full code.	26
5.3	Agentless search/replace edit format. Focused diffs encourage small patches and make candidate repairs easier to apply and validate.	28
5.4	Repair performance as the number of sampled patches increases. Sampling improves coverage, but returns plateau once patch ranking becomes the bottleneck. 28	
5.5	Example Agentless reproduction test. The generated test is an executable hypothesis about the issue, later used as evidence for patch selection.	29

5.6	Agentless reproduction-test workflow. Candidate tests are executed on the original repository and then used with regression tests to rank candidate patches.	30
5.7	Agentless audit of SWE-bench Lite. The categories reveal that benchmark items differ substantially in information quality, patch hints, and localization clues.	31
5.8	Solve-rate breakdowns on SWE-bench Lite-S. The filtered analysis makes task difficulty more visible than a single leaderboard score.	31

List of Tables

2.1	Representative benchmark families and the abilities they make visible. This table frames why SWE-bench is a shift from code generation toward software engineering.	6
2.2	Analytical dimensions used throughout this review. The dimensions make the three focal papers comparable even though one is a benchmark, one is an agent, and one is a pipeline.	6
3.1	Average and maximum attributes of SWE-bench task instances, adapted from the original benchmark statistics.	14
3.2	Model resolve rates with BM25 retrieval under different maximum context lengths in SWE-bench. Low rates show that lexical retrieval plus one-shot patching is insufficient for repository-level repair.	15
3.3	BM25 recall with respect to oracle files. The gap between retrieved context and oracle context explains why localization becomes a major research problem after SWE-bench.	15
3.4	Context-window comparison for oracle-style SWE-bench inputs. Longer context helps cover more instances, but it does not remove the need for focused localization.	15
4.1	Main SWE-agent results on SWE-bench and SWE-bench Lite. The ACI improves resolve rate over both static RAG and a shell-only agent, at higher inference cost.	21
4.2	Pass@1 results on HumanEvalFix. SWE-agent performs strongly on short debugging tasks, showing that its interface also supports simpler edit-and-fix workflows.	21
4.3	Selected ACI ablations on SWE-bench Lite. Performance drops when search, editing, file viewing, or context management is less aligned with LLM interaction patterns.	22
5.1	Performance of selected Agentless localization steps. Hierarchical localization reduces context size while preserving many ground-truth edit locations.	27
5.2	Performance of Agentless repair setups. Sampling patches over multiple localized contexts improves the final number of resolved issues.	28

5.3	Patch-validation ablation in Agentless. Generated reproduction tests provide the largest gain beyond majority voting and regression-test filtering.	29
5.4	Selected SWE-bench Lite results reported by Agentless. The table emphasizes that Agentless is competitive with agentic tools while remaining pipeline-based and relatively cheap.	30
5.5	Selected results on SWE-bench Lite-S, the filtered benchmark subset proposed by Agentless. Filtering removes tasks with exact patches, misleading solutions, or insufficient information.	31
6.1	Compact comparison of the three focal papers. The table summarizes the role each paper plays in the thesis’s benchmark-agent-pipeline narrative.	35
A.1	Supplementary comparison of evaluation assumptions across the three focal papers.	49

Chapter 1

Introduction

Large language models are no longer evaluated only as systems that complete a function body or answer a programming question. That older evaluation unit is too small for modern software engineering, because software engineering is a situated activity. A developer reads an issue report, identifies the relevant component, searches a repository, inspects previous code, runs tests, edits files, and interprets failures. These activities make the output requirement stricter than syntactic plausibility. The required output is a small, compatible, testable change inside an existing project. This repository-centered requirement changes the relevant ability from code generation to software maintenance.

This thesis studies that shift through three papers that form a connected research sequence in LLM-based automated software engineering. SWE-bench asks whether language models can resolve real-world GitHub issues under an executable evaluation protocol [15]. This protocol becomes the target for SWE-agent, which proposes an agent-based approach where a language model interacts with a repository through a specialized agent-computer interface, or ACI [50]. This agentic approach then becomes the contrast point for Agentless, which removes broad autonomous action and decomposes the same repair problem into localization, patch generation, and validation [48]. The three papers are therefore connected not only by shared experiments, but also by a shared systems question: how should a language model be embedded into the workflow of repository-level repair?

The three papers also connect to a much wider literature. Earlier code benchmarks such as HumanEval, MBPP, APPS, CodeContests, CodeXGLUE, and CodeSearchNet made code generation and code understanding measurable at scale [3, 6, 12, 14, 24, 27]. Language-model architectures and pretraining methods such as CodeT5, InCoder, CodeGen, and StarCoder expanded the capability of code-specialized models [9, 23, 33, 45]. At the same time, agentic reasoning and tool-use work such as ReAct, Toolformer, Reflexion, Voyager, WebArena, InterCode, AutoGen, ChatDev, and MetaGPT explored how language models can plan, call tools, and act in interactive environments [13, 37, 39, 40, 44, 47, 49, 51, 53]. SWE-bench, SWE-agent, and Agentless are best understood at the intersection of these two lines of work.

1.1 Research Questions

The thesis is organized around four research questions. First, what did SWE-bench change about the evaluation of language models for code? Second, why did SWE-agent argue that agent-computer interfaces are necessary for repository-level repair? Third, why did Agentless show that a non-agentic pipeline can compete with or outperform more complex agents on the same benchmark family? Fourth, what does the comparison imply for future software-engineering systems that must balance accuracy, cost, reliability, and human inspectability?

These questions are deliberately comparative. A paper-by-paper summary would miss the most important point: the papers define a shared problem and then explore different control strategies for that problem. SWE-bench turns a maintenance issue into a benchmark instance. SWE-agent treats the instance as an environment for interactive problem solving. Agentless treats the same instance as a structured repair pipeline. The relation among benchmark, agent, and pipeline is therefore the central object of analysis.

1.2 Repository-Level Code Repair

Repository-level repair is different from short-form code generation in several ways. It requires localization, because the system must decide which files and symbols matter. It requires compatibility, because a patch must fit existing APIs, tests, style, and dependency constraints. It requires validation, because the visible issue description may be incomplete and the repository may contain subtle hidden assumptions. It also requires process control, because a system must decide when to search, when to edit, when to run tests, and when to stop.

These requirements are familiar in classical automated program repair. Generate-and-validate repair, search-based repair, semantic repair, and spectrum-based fault localization all study how to identify and correct faulty code under test constraints [1, 11, 16, 19, 20, 26, 46]. However, language models change the problem by providing powerful natural-language and code-generation priors. They can read an issue description, infer intent, and synthesize edits that are not limited to a fixed mutation operator. The cost is that their behavior is probabilistic, context-sensitive, and sometimes difficult to audit. SWE-bench makes this new trade-off visible.

1.3 Selection of the Three Focal Papers

The three focal papers are selected because they form a clear conceptual chain. SWE-bench is the evaluation paper. It asks what a realistic benchmark should look like once code generation is no longer limited to single functions. SWE-agent is the agentic systems paper. It asks how an LLM should interact with files, commands, outputs, and edit feedback in order to solve the SWE-bench task. Agentless is the pipeline paper. It asks whether the apparent success of agents

depends on agentic autonomy or whether careful decomposition can achieve similar results with lower cost and higher interpretability.

The selection is also useful because the papers share enough context to support direct comparison. Agentless and SWE-agent both use the SWE-bench family of benchmarks. Both must solve the localization problem. Both must generate patches. Both must rely on tests or test-like evidence. Yet their control philosophy differs sharply. SWE-agent emphasizes interaction and flexible exploration. Agentless emphasizes staged decisions and reproducible artifacts. This makes the comparison meaningful rather than superficial.

1.4 Contributions of This Review

This thesis makes four contributions. First, it reorganizes the three papers into a single narrative about task definition, agentic interaction, and structured repair. Second, it develops a common vocabulary for comparing benchmark design, control structure, information flow, validation, and cost. Third, it relates the papers to the broader literatures on code benchmarks, autonomous agents, automated program repair, test generation, and benchmark validity. Fourth, it argues for a hybrid future in which pipelines and agents are not competing ideologies but complementary mechanisms.

The review also emphasizes operating-systems concerns that are sometimes implicit in machine-learning papers. Repository-level repair requires process isolation, command execution, file-system manipulation, dependency installation, sandboxing, and test orchestration. These concerns matter for both evaluation and deployment. An agent that can run arbitrary commands is powerful but risky. A pipeline that limits actions is easier to audit but may fail when the problem requires exploratory debugging. The systems perspective helps explain why the design of the environment is as important as the design of the model.

1.5 Methodological Posture

The thesis is a literature review, not a reproduction study. It does not rerun the original experiments. Instead, it reads the papers comparatively, using their reported designs, figures, tables, and stated limitations. The goal is not to declare one system universally better. The goal is to understand what each paper teaches about the design space of LLM-based software engineering.

This methodological choice is important because reported scores depend on benchmark version, model version, available context, inference budget, visible tests, and system implementation. Benchmark methodology research has repeatedly shown that headline results can hide important differences in setup and interpretation [10, 29, 30, 35, 42]. Therefore, this review treats performance numbers as evidence about design choices rather than as final rankings.

1.6 Chapter Outline

Chapter 2 provides background on code benchmarks, repository-level repair, agents, testing, and reproducibility. Chapter 3 analyzes SWE-bench as the common evaluation problem. Chapter 4 analyzes SWE-agent as an agent-based response to that problem. Chapter 5 analyzes Agentless as a pipeline-based response to the same problem. Chapter 6 synthesizes the comparison across benchmark definition, interaction, control, validation, and cost. Chapter 7 concludes with future work.

Chapter 2

Background and Preliminaries

2.1 From Code Completion to Software Engineering

The first wave of LLM4Code evaluation focused on whether models could synthesize short pieces of code. HumanEval asks a model to complete Python functions from docstrings [6]. MBPP uses short programming problems intended to be solved by simple functions [3]. APPS and CodeContests increase problem difficulty by using competitive-programming tasks, where the model must infer an algorithm and produce a complete solution [12, 24]. CodeXGLUE and CodeSearchNet broaden evaluation to retrieval, summarization, translation, and code understanding [14, 27]. These benchmarks were essential because they created shared tasks and measurable progress.

However, the assumptions of short-form benchmarks differ from the assumptions of software maintenance. In a short benchmark, the entire relevant context is usually contained in the prompt. The output is usually a function or file that can be tested independently. The problem statement is often written for a benchmark rather than for a real issue tracker. In repository-level repair, the relevant context may be spread across many files. The issue report may be incomplete, ambiguous, or written for maintainers who already know the codebase. The correct patch may require changing only a few lines, but finding those lines can be harder than writing the edit. SWE-bench addresses this gap by using real repositories and historical fixes.

The shift also changes the meaning of model capability. A model that scores well on HumanEval may know programming syntax and common algorithms, but it may still fail to navigate a repository or respect an existing API. A model that can solve APPS may reason about algorithms, but it may not know how to interpret a failing unit test in a mature codebase. Repository repair therefore requires a mixture of code understanding, natural-language understanding, search, planning, and environmental feedback.

Table 2.1: Representative benchmark families and the abilities they make visible. This table frames why SWE-bench is a shift from code generation toward software engineering.

Benchmark family	Representative examples	What the family primarily evaluates
Function synthesis	HumanEval, MBPP	Local code generation from compact prompts and unit tests
Algorithmic problem	APPS, CodeContests	Full-solution synthesis, algorithm selection, and test passing
Code understanding and retrieval	CodeXGLUE, CodeSearchNet	Semantic code search, summarization, translation, and representation learning
Repository-level repair	SWE-bench family	Issue understanding, localization, patch generation, environment execution, and validation

Table 2.2: Analytical dimensions used throughout this review. The dimensions make the three focal papers comparable even though one is a benchmark, one is an agent, and one is a pipeline.

Dimension	Guiding question
Task definition	What input, output, and success criterion does the paper establish?
Information flow	How does the system select, compress, or inspect repository context?
Control structure	Which decisions are delegated to the model, and which are fixed by the system?
Validation	What evidence is used to decide whether a patch resolves the issue?
Cost	What model, environment, budget, and logging assumptions shape the result?
Human inspectability	Can a developer review the intermediate artifacts and understand the repair path?

2.2 Repository-Level Repair as a Systems Problem

Repository-level repair is a systems problem because the model is only one component. The overall system includes a benchmark harness, a repository checkout, dependency management, test execution, prompts, retrieval, action interfaces, patch application, and result logging. Each component can affect the final score. If the environment is not reproducible, a correct patch may fail. If the retrieval system omits the relevant file, the model may never see the necessary context. If the edit interface is brittle, a plausible patch may be applied incorrectly. If tests are incomplete, a wrong patch may pass.

This systems view connects LLM4Code to classical program repair. GenProg explored search-based repair using genetic programming [20]. PAR studied human-written patch templates [19]. Angelix used semantic constraints to synthesize repairs [26]. PraPR studied practical mutation-based repair on real Java bugs [11]. Defects4J became a standard benchmark for Java bugs [17]. Surveys by Monperrus and Le Goues et al. emphasize that repair quality, test

adequacy, and benchmark construction are persistent issues [21, 31]. SWE-bench inherits these concerns but moves them into a modern LLM setting.

The systems perspective also explains why operating-systems topics are relevant. An automated repair system may execute test suites, spawn subprocesses, access files, install packages, and run commands generated by a model. Evaluation must isolate these actions. Deployment must prevent data leakage, accidental deletion, network misuse, and secret exposure. Agents make these concerns more urgent because they can choose commands interactively. Pipelines reduce some risk by narrowing the action space, but they still need reliable execution and sandboxing.

2.3 Fault Localization, Context Selection, and Retrieval

A central difficulty in repository-level repair is context selection. The model cannot usually read the entire repository in detail. It must identify candidate files, classes, functions, and lines. Classical fault localization studies how to use test spectra, coverage, suspiciousness metrics, and debugging information to rank likely faulty locations [1, 16, 32, 46]. LLM-based systems often use a different signal: natural-language similarity between the issue and code, model-based reasoning over file summaries, or interactive search.

SWE-bench begins with retrieval baselines such as BM25. This choice is simple but informative. If BM25 retrieves the relevant files, a model has a chance to patch them. If it retrieves unrelated files, the model may produce a plausible but irrelevant patch. SWE-agent replaces one-shot retrieval with interactive search and navigation. The agent can inspect files, use commands, and react to observations. Agentless takes a third path: it designs a hierarchical localization pipeline, moving from files to symbols to edit locations. These three approaches show that localization is not an implementation detail. It is a core part of the research problem.

Context selection is also linked to model architecture. Long-context models can accept more repository text, but more context is not always better. Extra files may distract the model or increase cost. Retrieval can improve focus but may miss implicit dependencies. An agent can search adaptively but may waste turns. A pipeline can be efficient but may become brittle if a step makes a wrong decision. Therefore, repository repair should report not only whether a patch passed but also how the system selected the information used to create it.

2.4 Agents, Tools, and Interfaces

Agentic language-model systems combine reasoning with actions. ReAct introduced a pattern in which a model interleaves reasoning traces and tool calls [51]. Toolformer showed that models can learn to use external tools [39]. Reflexion explored self-reflection over failed attempts [40]. Voyager studied open-ended skill acquisition in Minecraft [44]. WebArena and Mind2Web

evaluated web navigation agents [7, 53]. InterCode evaluated interactive coding and database tasks [49]. AutoGen, ChatDev, and MetaGPT explored multi-agent collaboration and software workflows [13, 37, 47]. SWE-agent belongs to this broader movement, but its environment is a real code repository rather than a synthetic tool task.

The key design issue is interface fit. Human interfaces are designed for people with visual perception, memory, and manual control. A raw shell gives a language model enormous flexibility, but it also creates many opportunities for inefficient or invalid behavior. The model may print too much output, lose track of file positions, edit the wrong lines, or repeat commands. SWE-agent argues that an agent-computer interface should be designed for the strengths and weaknesses of language models. This is analogous to human-computer interaction, but the user is an LLM.

Agentless challenges the assumption that more autonomy is always better. It uses LLMs, but it does not let them freely choose the next action in an open-ended loop. The system designer specifies the phases. This difference matters because autonomy affects cost, safety, reproducibility, and interpretability. An agent can recover from unexpected observations, but its trajectory may be long and hard to compare. A pipeline may be easier to analyze, but it may fail when the issue requires a novel debugging path.

2.5 Testing, Validation, and Benchmark Validity

Tests are the main bridge between code generation and software engineering evaluation. A generated patch is not merely judged by appearance; it is applied to a repository and evaluated by tests. This principle connects SWE-bench to automated program repair, regression testing, and test generation. Tools such as EvoSuite and Randoop generate tests automatically [8, 34]. Fuzzing research studies large-scale input generation for finding bugs [28, 52]. Recent work has also used LLMs for unit test generation and bug reproduction [22, 43]. Agentless uses reproduction tests as part of patch validation, which shows how test generation becomes a component of repair rather than only a final evaluation.

Test-based evaluation is powerful but imperfect. If hidden tests capture the issue, passing them is a meaningful success signal. If hidden tests are incomplete, a patch may overfit. If the historical pull request contained broad refactoring, the expected tests may not isolate the essential behavior. If the environment is flaky, results may depend on setup rather than patch quality. SWE-bench makes these issues visible, and Agentless explicitly audits part of the SWE-bench Lite set to create SWE-bench Lite-S. This audit is important because benchmark quality is itself a research object.

Benchmark validity also depends on contamination and temporal effects. A model may have seen code or issues during pretraining. SWE-agent discusses issue age and performance to examine possible test pollution. SWE-bench uses real projects, which improves realism but also

makes contamination harder to exclude completely. The correct response is not to abandon real benchmarks, but to report dates, splits, repository versions, and evaluation details clearly.

2.6 Cost, Reproducibility, and Scientific Claims

Cost is a first-class constraint in repository-level repair. Costs include model calls, tokens, environment setup, test execution time, and developer time spent inspecting outputs. SWE-agent can spend many turns interacting with the environment. Agentless can generate multiple candidate patches and tests, but its stages are more bounded. A benchmark score without a cost profile is incomplete. A system that solves slightly more issues at ten times the cost may be valuable in some settings and impractical in others.

Reproducibility is also difficult. Proprietary model versions change. Dependencies become unavailable. Repositories evolve. Test suites may rely on external services or timing assumptions. Benchmark methodology work recommends richer reporting, model cards, datasheets, and reproducibility standards [10, 30, 35]. In this thesis, these recommendations are applied to automated software engineering: papers should report model versions, prompt templates, cost limits, candidate counts, visible tests, benchmark versions, and failure categories.

The broader lesson is that LLM-based software engineering should be evaluated as a socio-technical system. It combines models, data, interfaces, tests, and developer expectations. SWE-bench, SWE-agent, and Agentless each illuminate one part of that system. The rest of the thesis studies them in detail.

2.7 Review Approach and Broader Literature

This review focuses on three papers because they form a tight methodological chain around repository-level repair. The scope is therefore deliberately narrower than the full LLM4Code literature. It does not attempt to rank every SWE-bench solver, reproduce every experiment, or evaluate commercial systems whose trajectories and prompts are unavailable. Instead, it uses the three focal papers to study how benchmark construction, agent-computer interfaces, and structured repair pipelines divide responsibility between the model and the surrounding system.

This scope also limits the claims that can be made. The thesis compares reported designs and results rather than rerunning systems under one unified harness. Differences in model versions, benchmark splits, prompt templates, inference budgets, and visible test access may affect reported performance. For this reason, the comparison emphasizes design trade-offs and evaluation methodology rather than treating any leaderboard number as an absolute measure of intelligence.

Cross-paper comparison is difficult because each paper answers a different primary question. SWE-bench asks how to define a realistic benchmark; SWE-agent asks how an LLM should

interact with a repository; Agentless asks how much autonomy is needed after the task is decomposed. Comparing them as if they were three identical solvers would be misleading.

There are also practical threats. Benchmark versions evolve, public repositories may appear in pretraining data, proprietary model endpoints change, and hidden test suites can encode assumptions that are not visible from the issue text. Cost comparisons are also sensitive to pricing and stopping rules. The review treats these threats as part of the systems lesson: credible automated software engineering requires not only high accuracy, but also transparent task definitions, reproducible environments, and auditable intermediate evidence.

The three focal papers are recent, but their questions are rooted in a longer history of software engineering and machine learning for code. The naturalness hypothesis argued that source code has statistical regularities that can be modeled, and later work showed that those regularities extend to defects, edits, and developer intent [2, 38]. This earlier perspective matters because LLM4Code systems inherit a basic assumption from big-code research: code is not arbitrary text, but a structured artifact with conventions, repeated idioms, and project-specific patterns. SWE-bench makes this assumption harder to exploit because the model must operate inside full repositories, but it does not abandon it. The ability to infer likely edit locations or likely patch shapes is still partly a learned regularity over code.

The benchmark literature before SWE-bench also shows a gradual expansion of task realism. HumanEval and MBPP test function-level generation [3, 6]. APPS and CodeContests test algorithmic problem solving under unit tests [12, 24]. MultiPL-E tests whether generation behavior transfers across programming languages, while EvalPlus shows that weak tests can overestimate model correctness [5, 25]. These works are not competitors to SWE-bench; they are stepping stones. They isolate important abilities, such as syntax, algorithmic reasoning, and test passing. SWE-bench combines several of those abilities with repository navigation and historical issue resolution. This is why SWE-bench can be read as a benchmark that moves from code generation toward software engineering.

The automated program repair literature provides another foundation. The “plastic surgery” hypothesis suggests that useful repairs can often be assembled from code already present in a project [4]. This idea helps explain why retrieval and localization are so important for LLM repair. If the needed behavior resembles existing code, then the main challenge is to find and adapt the right pattern. Conversely, if the relevant pattern is absent or scattered across files, a solver needs deeper reasoning and validation. SWE-agent’s search behavior and Agentless’s file skeletons can both be interpreted as mechanisms for finding reusable project-local evidence.

Benchmark validity and leakage research adds caution. Modern models are trained on large web-scale corpora, so public repositories and public issue discussions may be present in training data. Leakage can make results appear stronger than they are, especially when evaluation items are drawn from public sources [18]. This does not make public benchmarks useless. Rather, it means that benchmark papers should report construction dates, repository commits, model

versions, temporal analyses, and contamination checks. SWE-agent’s discussion of issue age and Agentless’s benchmark audit both move in this direction. A literature review should therefore treat benchmark performance as an empirical signal that needs context, not as an absolute measure of intelligence.

The agent literature adds a different caution. Many agent papers show impressive demonstrations, but software engineering requires more than task completion in a sandbox. It requires reproducible edits, reviewable patches, and safe execution. A web-navigation agent can sometimes recover from a wrong click; a code-repair agent can damage a repository or generate a patch that passes tests for the wrong reason. SWE-agent is valuable because it makes the interface explicit, while Agentless is valuable because it asks how much of the action loop can be replaced by deterministic control. Together, they suggest that the agent concept should be decomposed. One should ask what the model chooses, what the system chooses, what evidence is observed, and what constraints prevent harmful behavior.

Finally, the testing literature explains why validation remains central even as models improve. Test generation tools such as EvoSuite and Randoop show that automatically produced tests can expose behavior, but generated tests are themselves artifacts that require interpretation [8, 34]. Fuzzing surveys show that input generation can find bugs at scale, but also that oracles and triage are difficult [28]. Agentless’s reproduction tests fit this tradition. They are not simply extra prompts; they are attempts to create executable evidence. The success of future LLM repair systems will depend on how well they combine natural-language issue understanding with executable validation evidence.

2.8 Summary

This chapter establishes the vocabulary used in the rest of the thesis. Earlier benchmarks measure important but narrower abilities, while repository-level repair combines issue understanding, context selection, patching, execution, and validation. The chapter also identifies the main evaluation risks—test adequacy, contamination, cost, and reproducibility—that reappear in the three focal papers. With this background in place, Chapter 3 turns to SWE-bench as the benchmark that makes these repository-level requirements measurable.

Chapter 3

SWE-bench: Establishing the Common Evaluation Problem

3.1 The Problem SWE-bench Defines

SWE-bench begins from a simple but consequential observation: the next frontier for code models is not only writing new functions, but modifying existing repositories. A real issue report rarely gives the exact function to edit. It may mention a symptom, an error message, or a behavior that should change. The developer must connect that text to code. SWE-bench converts this situation into a benchmark by collecting GitHub issues and pull requests from real Python repositories. Each benchmark instance gives a model an issue description and a repository snapshot. The model must produce a patch, and the patch is evaluated by running tests associated with the historical fix [15].

This benchmark definition is the foundation for both SWE-agent and Agentless. Without a shared task, it would be difficult to compare an agentic system and a structured pipeline. SWE-bench supplies the common problem: repository-level issue resolution under executable evaluation. The task is realistic because the issue text, codebase, and patches come from actual software maintenance. It is measurable because the model’s output is applied and tested. It is difficult because the system must solve localization, understanding, editing, and validation together.

SWE-bench also changes the interpretation of failure. In short code benchmarks, failure often means the generated function does not satisfy tests. In SWE-bench, failure may mean the system did not find the right file, misunderstood the issue, exceeded context limits, produced an invalid patch, broke unrelated behavior, or encountered an environment problem. This richer failure space is precisely why the benchmark is useful. It forces researchers to build systems, not only prompts.



Figure 3.1: SWE-bench collection pipeline. Real GitHub issues are linked to merged pull requests, installable repository snapshots, and executable tests, turning software maintenance into a benchmarkable task.

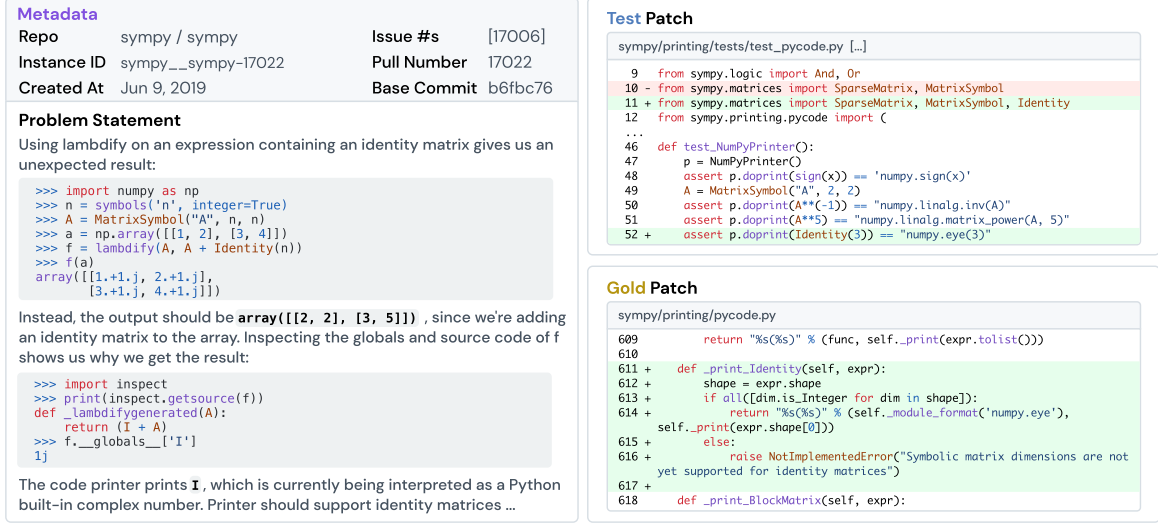


Figure 3.2: Example SWE-bench task instance. The issue text, repository snapshot, reference patch, and tests define a repair problem whose solution must be executable rather than merely plausible.

3.2 Benchmark Construction and Task Representation

The construction of SWE-bench links issue reports to merged pull requests. The benchmark uses the repository state before the pull request as the input state and uses tests associated with the fix to evaluate candidate patches. This design is stronger than synthetic bug injection because the issue and fix were created by real maintainers. It also gives each task a natural-language component and an executable component. The natural-language issue describes what is wrong. The tests provide an operational definition of whether a patch resolves the problem.

A SWE-bench task instance therefore contains several kinds of information: the problem statement, the repository identifier, the base commit, the reference patch, and test specifications. The reference patch is not given to the solver, but it is used to construct and analyze the task. The solver receives the issue and repository, then outputs a patch. This representation matters because it separates the developer-facing input from the evaluator-facing ground truth. A system must infer the fix from the same kind of information that a human maintainer might initially see.

The benchmark includes thousands of task instances across multiple repositories. Table-level statistics in the original paper show that tasks vary in issue length, patch size, repository size, and test structure. This variation is important because a benchmark with only small or uniform tasks could be solved by narrow heuristics. SWE-bench instead includes a range of maintenance

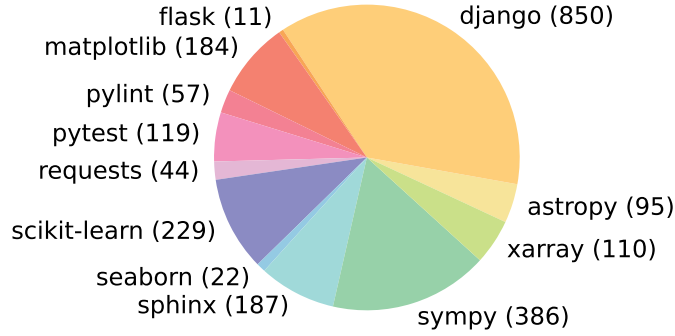


Figure 3.3: Distribution of SWE-bench tasks across the twelve source repositories. The spread across projects makes localization and repository-specific compatibility central parts of the task.

Table 3.1: Average and maximum attributes of SWE-bench task instances, adapted from the original benchmark statistics.

Category	Attribute	Mean	Max
Issue text	Length (words)	195.1	4,477
Codebase	# non-test files	3,010	5,890
Codebase	# non-test lines	438K	886K
Gold patch	# lines edited	32.8	5,888
Gold patch	# files edited	1.7	31
Gold patch	# functions edited	3.0	36
Tests	# fail-to-pass tests	9.1	1,633
Tests	# total tests	120.8	9,459

problems, from localized bug fixes to changes that require understanding interactions across files.

The task representation also creates a strong context-selection challenge. If a repository contains thousands of lines of code, the model cannot simply read everything with equal attention. The benchmark therefore exposes the importance of retrieval and summarization. A solver must decide what to show the model and in what order. This is why SWE-bench is not merely a code-generation benchmark; it is also an information-access benchmark.

3.3 Retrieval, Context Length, and Baseline Solvers

The SWE-bench paper evaluates baseline solvers that retrieve files and ask language models to generate patches. BM25 retrieval is a natural starting point because it is simple, reproducible, and based on lexical similarity between the issue and repository files. The baseline reveals an important limitation: even strong models struggle when the relevant context is missing or diluted by irrelevant files. Increasing context length does not automatically solve the problem, because longer prompts may contain more noise and cost more to process.

Table 3.2: Model resolve rates with BM25 retrieval under different maximum context lengths in SWE-bench. Low rates show that lexical retrieval plus one-shot patching is insufficient for repository-level repair.

Model	Maximum context		
	13k	27k	50k
Claude 2	1.96	1.87	1.22
SWE-Llama 7B	0.70	0.31	0.00
SWE-Llama 13B	0.70	0.48	0.00

Table 3.3: BM25 recall with respect to oracle files. The gap between retrieved context and oracle context explains why localization becomes a major research problem after SWE-bench.

Metric	BM25 recall		
	13k	27k	50k
Average	29.58	44.41	51.06
All oracle files	26.09	39.83	45.90
Any oracle file	34.77	51.27	58.38

Table 3.4: Context-window comparison for oracle-style SWE-bench inputs. Longer context helps cover more instances, but it does not remove the need for focused localization.

	ChatGPT-3.5	GPT-4	Claude 2	SWE-Llama
Maximum tokens	16,385	32,768	100,000	$\geq 100,000$
Instances covered	58.1%	84.1%	96.4%	$\geq 94.8\%$

SWE-bench also studies an oracle retrieval setting. In that setting, the solver receives files edited by the reference patch. Oracle retrieval is not realistic because a developer does not know the correct files in advance, but it is analytically useful. It separates localization from patch generation. If a model performs much better with oracle files, then localization is a major bottleneck. If performance remains low even with oracle files, then patch synthesis and reasoning are also difficult. This distinction is crucial for comparing later systems. SWE-agent and Agentless can both be understood as attempts to improve the localization and context-selection parts of the problem.

The baseline results show that the benchmark is difficult for contemporary models. This difficulty is not a weakness; it is the point. A benchmark should reveal unsolved capabilities. SWE-bench shows that repository-level repair requires better retrieval, better interfaces, better validation, and better repair strategies. It creates room for the research contributions of SWE-agent and Agentless.

3.4 Evaluation Strengths and Validity Risks

The strongest feature of SWE-bench is executable evaluation. A patch is not graded only by natural-language judgment. It is applied to the repository and tested. This makes results more objective than manual assessment of code plausibility. It also aligns evaluation with real software engineering, where code changes must run.

However, executable evaluation is only as strong as the tests and environment. Tests may be incomplete. A patch may pass hidden tests while still being undesirable. Some issues may require design judgment not captured by tests. Environment setup can be fragile, especially for older commits. Dependencies may change or disappear. These risks do not invalidate SWE-bench, but they mean that resolve rate should be interpreted carefully.

Another validity issue is the relationship between historical patches and current model training data. Because SWE-bench uses public GitHub repositories, there is a possibility that models have seen some code, issues, or patches during pretraining. The SWE-bench paper and subsequent work discuss temporal splits and contamination concerns, but perfect exclusion is hard in open web data. This again supports the literature-review stance of this thesis: a score is informative only when paired with details about data, model, time, and evaluation procedure.

SWE-bench’s design also raises a question about what counts as a successful repair. Passing tests is necessary for benchmark success, but human maintainers also care about readability, minimality, compatibility, and maintainability. Classical program-repair literature has long warned about overfitting to tests and generating plausible but semantically wrong patches [36, 41]. LLM-based repair inherits the same problem. SWE-bench makes progress measurable, but it does not eliminate the need for human review.

3.5 Relationship to SWE-agent and Agentless

The importance of SWE-bench becomes clearer when reading SWE-agent and Agentless. SWE-agent uses SWE-bench to demonstrate that interactive agents can outperform non-interactive retrieval baselines. Agentless uses the SWE-bench Lite family to show that a structured pipeline can achieve strong results without an open-ended agent loop. In both cases, the benchmark provides the task definition and the evaluation target.

The relation is not merely chronological. SWE-bench defines the object of optimization. SWE-agent changes the action model: instead of receiving retrieved files once, the model can search, inspect, and edit interactively. Agentless changes the control model: instead of letting the model decide each next step, the system decomposes repair into explicit phases. Thus, SWE-bench is the shared experimental ground on which different philosophies of LLM software engineering can be compared.

This also means that benchmark improvements affect solver design. If future SWE-bench versions include more languages, more complex build systems, or richer human feedback, the

relative advantage of agents and pipelines may change. A pipeline tuned for Python unit-test repair may need new modules for distributed systems. An agent may need safer sandboxing or domain-specific tools. The benchmark and solver co-evolve.

3.6 Summary and Transition to SWE-agent

SWE-bench is a turning point because it changes the research question from “can a model write code?” to “can a model participate in software maintenance?” The difference is large. Software maintenance includes understanding existing code, respecting constraints, and validating behavior. It is also economically important, because much engineering time is spent modifying existing systems rather than writing isolated programs from scratch.

The benchmark also encourages more honest system evaluation. A system that works only when given the correct file is not yet a full repair system. A system that generates a patch but cannot apply it is incomplete. A system that passes tests but costs too much may be impractical. By forcing all these concerns into one task, SWE-bench makes LLM4Code research more systems-oriented.

The next two chapters study two responses to this benchmark. This benchmark first motivates SWE-agent, because a repository-level task requires an interface through which a model can search, edit, and test. This interface-centered response then motivates the contrast with Agentless, because once the task and artifacts are explicit, the repair process can be decomposed into stages. Both responses are possible because SWE-bench provides a shared definition of success.

A more detailed reading of SWE-bench also shows why the benchmark is a bridge between dataset construction and systems engineering. The dataset is not simply a table of prompts and answers. Each instance requires a repository checkout at a particular commit, a way to apply model patches, a mechanism for selecting tests, and a runner that can decide whether behavior matches the historical fix. These details make the benchmark much closer to a continuous-integration workload than to a text-only benchmark. The benchmark harness must therefore manage process execution, dependency installation, timeouts, and patch failures. A solver can fail before semantic evaluation if its patch cannot be applied cleanly. This means that SWE-bench measures the whole path from natural language to executable repository state.

The benchmark also makes the granularity of repair visible. Some issues require a one-line condition change. Others require modifying multiple files or adjusting tests. Some issue reports mention the relevant function directly, while others describe a symptom that must be traced through the project. This variety is important because different solvers may specialize in different parts of the distribution. A retrieval baseline may work when the issue text shares vocabulary with a relevant file. An agent may work when evidence must be gathered dynamically. A pipeline may work when structural localization can narrow the search. Reporting only the aggregate resolve rate hides these subpopulations.

Another important point is that SWE-bench makes negative results productive. Low solve rates are not merely disappointing; they identify missing system capabilities. If BM25 recall is weak, then retrieval and localization need improvement. If oracle-file performance is still low, then patch reasoning needs improvement. If patches apply but fail tests, validation and semantic understanding need improvement. If patches do not apply, edit representation and patch formatting need improvement. In this way, SWE-bench provides a decomposition of research questions even though the benchmark itself reports an end-to-end task.

Chapter 4

SWE-agent: A Classic Agent-Based Approach

4.1 From Static Prompting to Interactive Repair

SWE-agent begins with a critique of static prompting for repository repair. A one-shot prompt must decide in advance which files to retrieve and how much context to include. If retrieval is wrong, the model has little chance to recover. Human developers do not work this way. They inspect, search, test, and revise. SWE-agent therefore treats repository repair as an interactive task in which a language model acts inside a computer environment [50].

The key contribution is the agent-computer interface. The paper argues that simply giving a model a raw shell is not enough. A shell is powerful, but it is designed for humans who can manage screen output, remember context, and correct manual mistakes. Language models have different failure modes. They may generate malformed commands, forget line numbers, repeatedly inspect irrelevant files, or fail to notice that an edit did not apply. SWE-agent designs interface components that make repository navigation and editing more suitable for an LLM.

This is the agent-based response to SWE-bench. SWE-bench defines the issue-resolution task. SWE-agent asks how a model should act while solving it. The model is not only a patch generator; it is a decision-making component that observes outputs, chooses actions, and updates its plan. This approach fits the broader agent literature, but the software-engineering setting adds concrete constraints: commands must be safe, edits must be valid, and tests must be interpreted.

4.2 Agent-Computer Interface Design

The ACI is designed around the idea that an LLM needs a different interface from a human developer. Human IDEs such as VSCode and PyCharm provide rich visual and interactive affordances. A language model consumes text and produces text. Therefore, the interface

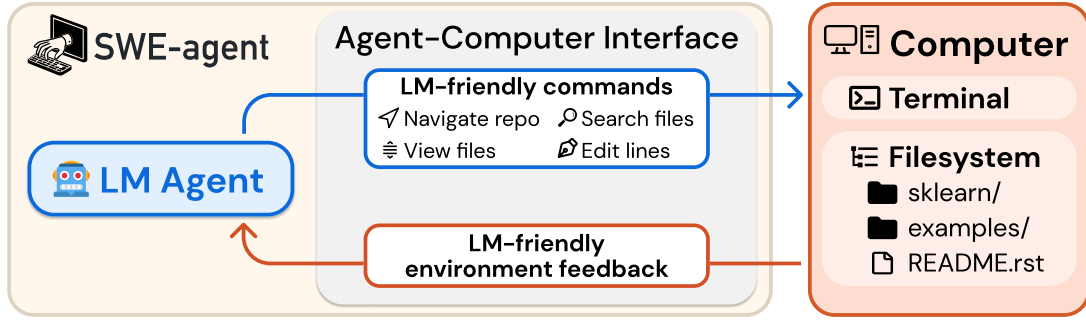


Figure 4.1: SWE-agent overview. The model repeatedly observes repository state, selects interface actions, edits files, and submits the resulting repository state as a patch.

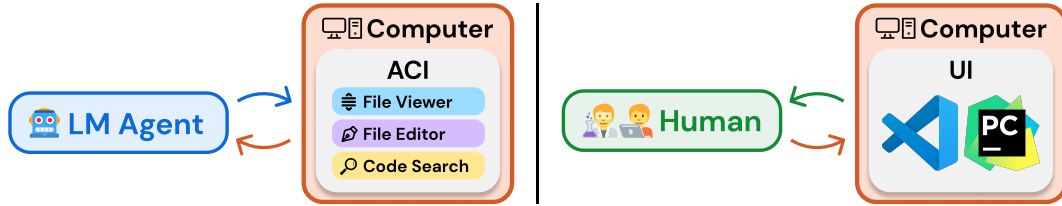


Figure 4.2: Agent-computer interface design. SWE-agent treats the textual interface as part of the system, not as a neutral wrapper around the model.

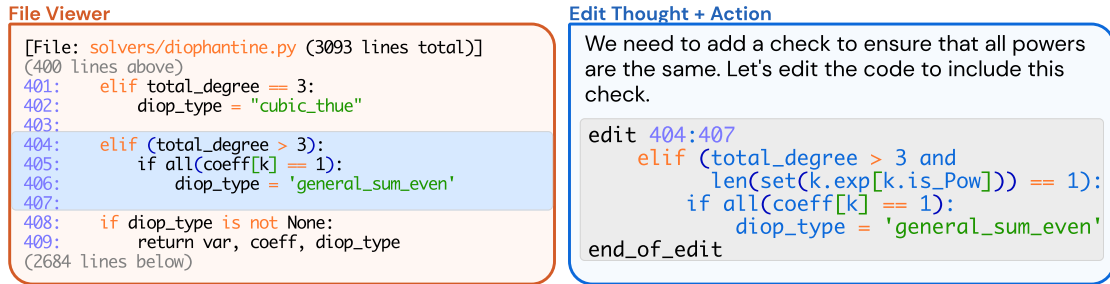


Figure 4.3: SWE-agent file-viewing and editing interface. Controlled windows and explicit edit feedback reduce the brittleness of raw shell-based editing.

must translate repository state into concise textual observations and translate model actions into reliable file operations.

One important component is the file viewer. Instead of dumping entire files, the interface shows controlled windows of code and tracks locations. This reduces context waste and helps the model maintain a local view of the repository. Another component is the edit command. Editing by line numbers in a raw shell can be brittle. SWE-agent integrates viewing and editing so that the model receives feedback about whether the edit applied. Guardrails help prevent silent failure.

The interface also manages history and errors. If a model produces malformed output, the environment returns an error message and asks for a valid action. If earlier observations become too long, history processors collapse them. These choices are not cosmetic. They determine how many useful turns fit in the context window and how quickly the model can recover from

Table 4.1: Main SWE-agent results on SWE-bench and SWE-bench Lite. The ACI improves resolve rate over both static RAG and a shell-only agent, at higher inference cost.

Setting	SWE-bench		SWE-bench Lite	
	% resolved	Avg. cost	% resolved	Avg. cost
RAG with GPT-4 Turbo	1.31	\$0.13	2.67	\$0.13
RAG with Claude 3 Opus	3.79	\$0.25	4.33	\$0.25
Shell-only with GPT-4 Turbo	—	—	11.00	\$1.46
SWE-agent with GPT-4 Turbo	12.47	\$1.59	18.00	\$1.67
SWE-agent with Claude 3 Opus	10.46	\$2.59	13.00	\$2.18

Table 4.2: Pass@1 results on HumanEvalFix. SWE-agent performs strongly on short debugging tasks, showing that its interface also supports simpler edit-and-fix workflows.

Model	Python	JavaScript	Java
CodeLLaMa-Instruct-13B	29.2	19.5	32.3
GPT-4	47.0	48.2	50.0
DeepSeekCoder-CodeAlpaca-6.7B	49.4	51.8	45.1
WaveCoder-DS-6.7B	57.9	52.4	57.3
SWE-agent with GPT-4 Turbo	87.7	89.7	87.9

mistakes. In repository repair, context management is equivalent to memory management for the agent.

4.3 Evaluation on SWE-bench and Related Tasks

SWE-agent evaluates on the full SWE-bench test set and the SWE-bench Lite split. It compares against retrieval-augmented generation baselines and a Shell-only interactive baseline. The main result is that an LM-friendly ACI improves performance substantially compared with both static retrieval and raw shell interaction. This supports the paper’s central claim: interface design changes agent performance.

The paper also evaluates on HumanEvalFix, a smaller debugging benchmark. This evaluation is useful because it separates basic editing ability from repository-scale complexity. Strong performance on HumanEvalFix indicates that the agent can perform simpler debugging tasks, while SWE-bench performance shows the harder repository-level setting.

The cost numbers are especially important. SWE-agent can be much more expensive than static retrieval because it uses multiple interaction turns and model calls. The paper reports cost alongside performance, which makes the trade-off explicit. An agent may solve more tasks, but it pays for exploration. This cost-performance relationship is one of the main themes in Chapter 6.

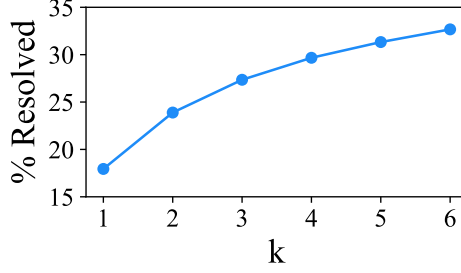


Figure 4.4: SWE-agent pass@k behavior across repeated runs on SWE-bench Lite. The curve shows that repeated agent trajectories can solve partially different instances.

Table 4.3: Selected ACI ablations on SWE-bench Lite. Performance drops when search, editing, file viewing, or context management is less aligned with LLM interaction patterns.

Component	Setting	% resolved
Editor	edit action	15.0
Editor	edit action with linting	18.0
Editor	no edit action	10.3
Search	summarized search	18.0
Search	iterative search	12.0
Search	no search	15.7
File viewer	30-line window	14.3
File viewer	100-line window	18.0
File viewer	full file	12.7
Context	last five observations	18.0
Context	full history	15.0
Context	without demonstration	16.3

4.4 Ablations, Behavior, and Failure Modes

A major strength of the SWE-agent paper is that it does not report only a final score. It studies interface ablations and agent behavior. The ablation table shows that components of the ACI matter. Removing or changing interface features can reduce performance. This suggests that the agent’s success is not only a property of the base model, but also a property of the environment in which the model acts.

The paper also analyzes action patterns, turns, failed edits, and failure modes in its appendix. These analyses are valuable because agent traces can be difficult to interpret. A trajectory may contain many searches, file views, failed edits, and test commands. Without categorization, it is hard to know whether the agent failed because it localized the wrong file, misunderstood the code, ran out of budget, or made a mechanical edit error. SWE-agent’s behavioral analysis begins to turn trajectories into diagnostic artifacts.

The failure modes also show why raw autonomy is risky. An agent may spend too many turns exploring irrelevant files. It may run commands that produce large outputs. It may become stuck after a failed edit. It may submit a partial fix when the budget is exhausted. These behaviors do

not mean agents are useless. They mean that agents require careful interface design, stopping rules, and logging. From a systems perspective, the ACI is a control surface.

4.5 Strengths, Limitations, and Scientific Position

SWE-agent’s main strength is that it makes interaction concrete. Many discussions of LLM agents are abstract. SWE-agent shows exactly how interface choices such as file viewing, editing, history processing, and error messages affect software repair. This is a significant contribution because it moves the field from prompt-only methods toward environment-aware systems.

Another strength is that SWE-agent fits human intuition about debugging. Developers rarely fix issues by reading a static retrieved bundle once. They explore. They form hypotheses. They inspect definitions. They run commands. An agentic system can approximate this workflow. When the problem requires discovering hidden dependencies or interpreting test failures, interactive exploration may be essential.

The limitations are equally important. First, the approach is costly. Each turn may require a large model call. Second, reproducibility depends on model version, prompt details, interface implementation, and environment setup. Third, agent traces can be long and hard to audit. Fourth, autonomy increases safety concerns because commands may execute code or access files. Finally, the best interface may depend on language, framework, and repository style.

SWE-agent therefore occupies a specific position in the literature. It is not simply a higher score on SWE-bench. It is an argument that the interface between an LLM and a development environment is a research object. It extends agentic tool-use ideas into realistic software engineering and shows that interface design is as important as model selection.

4.6 Summary and Transition to Agentless

SWE-agent depends on SWE-bench because the benchmark defines the problem that the agent solves. At the same time, SWE-agent reveals what SWE-bench demands from solvers. The benchmark is not only testing code synthesis; it is testing whether a system can gather information, operate on files, and use feedback. SWE-agent makes these hidden requirements explicit.

Agentless later responds to SWE-agent in an indirect but important way. It asks whether the same benchmark success requires broad interaction. If a pipeline can localize, generate, and validate patches without an open-ended action loop, then some benefits attributed to agency may actually come from decomposition, retrieval, and validation. This does not invalidate SWE-agent. It clarifies the design space. The field needs to know when interaction is worth its cost and when a structured pipeline is enough.

The comparison suggests a practical rule. Use agents when the problem requires exploration, uncertain context, or iterative debugging. Use pipelines when the problem can be decomposed into reliable stages. The best future systems may combine both strategies rather than choosing one permanently.

A further lesson from SWE-agent is that agents need explicit support for state. Human developers maintain state through memory, editor tabs, visual layout, and working notes. A language model receives a serialized text history. If the history grows too long, old evidence may be truncated or compressed. If it is too short, the model may repeat previous mistakes. SWE-agent’s history processing is therefore not a minor engineering trick. It is part of the cognitive architecture of the agent. The system decides what the model remembers, what it forgets, and what evidence remains salient.

The ACI also demonstrates that tool design can shape reasoning. A search command that returns too many matches may overwhelm the model. A file viewer that lacks surrounding context may cause the model to miss dependencies. An edit command that silently fails can corrupt the trajectory because the model will reason as if the change happened. These interface failures are analogous to bad APIs for human programmers. They do not merely slow the user; they change what actions are feasible. SWE-agent’s contribution is to make these interface properties measurable through benchmark performance and ablation.

SWE-agent’s cost profile should also be interpreted carefully. High cost is not only a financial issue. It changes system behavior. A tight per-instance budget encourages early submission and discourages broad exploration. A generous budget may improve success but also increase the chance of wandering. In human debugging, time pressure has similar effects. Therefore, the budget is part of the task definition. Future agent papers should treat token budget, turn budget, and command budget as experimental variables rather than hidden implementation details.

The paper also points toward a richer notion of agent evaluation. A successful trajectory may still be inefficient, and a failed trajectory may contain useful diagnostic information. For example, an agent might correctly identify the faulty file but produce the wrong patch. Another might produce a useful reproduction command but fail to edit. These partial successes matter in a developer-assistance setting. They also matter scientifically, because they reveal which components need improvement. SWE-agent’s traces make such analysis possible, while a one-shot patch generator usually leaves fewer intermediate artifacts.

Chapter 5

Agentless: A Classic Pipeline-Based Approach

5.1 Why an Agentless Approach Matters

Agentless begins from a provocative question: are autonomous software-engineering agents necessary, or can a simpler pipeline solve many of the same tasks? The paper does not reject language models. Instead, it rejects the assumption that an LLM must control an open-ended loop of actions. Agentless uses LLMs inside well-defined stages: localization, repair, and patch validation [48]. This makes it a pipeline-based response to the same repository-level repair problem established by SWE-bench.

The importance of this question is practical and scientific. Practically, agents can be expensive, slow, and hard to audit. Scientifically, if a simpler pipeline performs well, then researchers must be careful about attributing success to agency itself. The performance may come from good localization, multiple patch candidates, strong validation, or better use of tests. Agentless therefore acts as a structured counterpoint to SWE-agent.

The pipeline philosophy also aligns with classical program repair. Many repair systems generate candidate patches and validate them against tests. Agentless updates this pattern for LLMs. The model supplies flexible reasoning and patch generation, but the system designer controls the workflow. This combination reduces some risks of unconstrained autonomy while preserving much of the generative power of modern models.

5.2 Hierarchical Localization

The first stage of Agentless is localization. The system attempts to identify where a patch should be made. It does this hierarchically, moving from broad repository-level information to more precise edit locations. The paper discusses file-level localization, class or function localization,

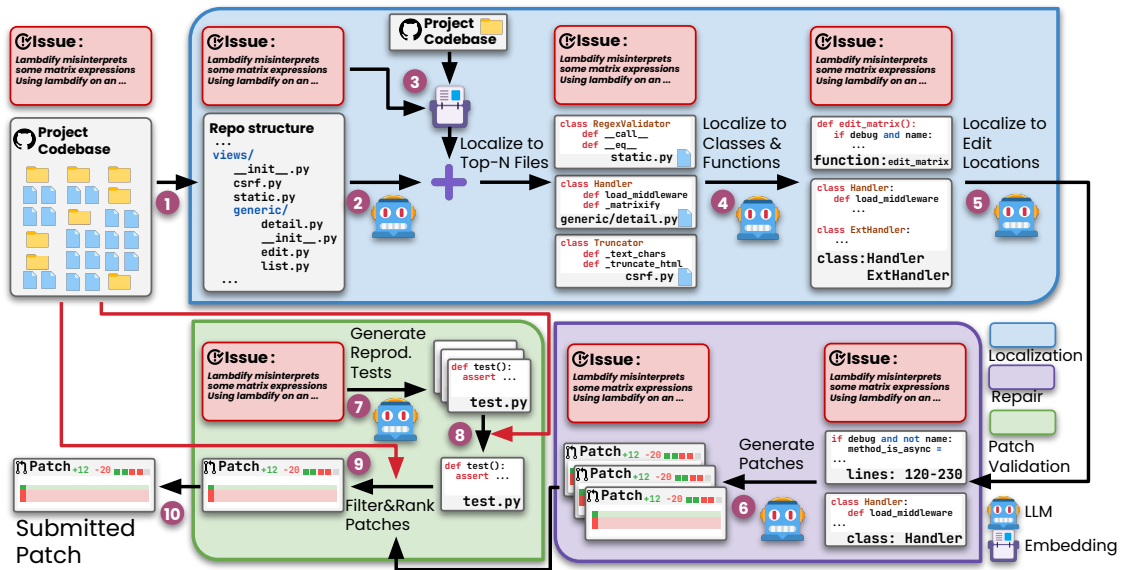


Figure 5.1: Agentless overview. The repair process is decomposed into localization, repair, and patch validation instead of being delegated to an open-ended agent loop.

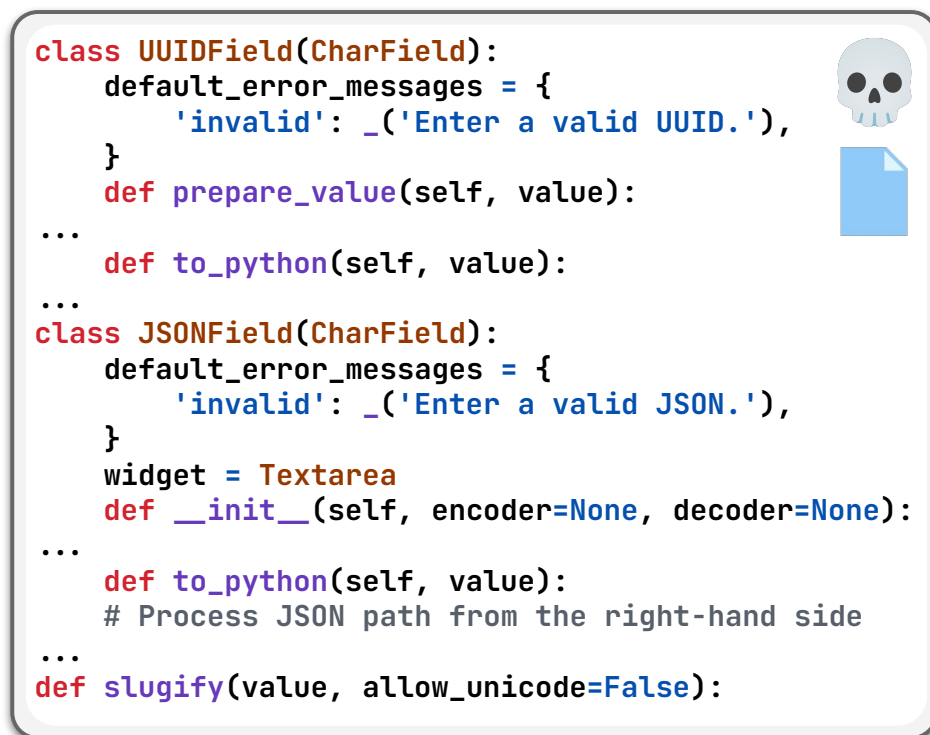


Figure 5.2: Agentless file skeleton format. The skeleton compresses file content into declarations so the model can localize relevant classes and functions before seeing full code.

and line-level localization. This staged design reflects the fact that repair quality depends heavily on finding the right code.

The file skeleton representation is important because it compresses repository structure. A model cannot always inspect full files in detail, but it can reason over a skeleton that lists classes, functions, and relevant signatures. This gives the model a map of the codebase without

Table 5.1: Performance of selected Agentless localization steps. Hierarchical localization reduces context size while preserving many ground-truth edit locations.

Stage	Method	Contains GT	Avg. cost
File level	Prompting-based	78.67%	\$0.02
File level	Embedding-based with filtering	70.33%	\$0.04
File level	Combined	81.67%	\$0.06
Related elements	Complete file	53.67%	\$0.15
Related elements	Skeleton format	58.33%	\$0.02
Edit location	Greedy	50.67%	\$0.06
Edit location	Multi-samples merged	56.33%	\$0.07

consuming the full context budget. Compared with raw retrieval, hierarchical localization is more structured. Compared with an agent, it is more bounded.

Localization is also where Agentless most directly challenges agentic approaches. An agent can search interactively, but it may do so inefficiently. A pipeline can ask targeted localization questions and produce ranked candidates. If the ranking is good, later stages can focus on a small edit region. If the ranking is bad, the pipeline may fail before repair begins. Thus, localization is both the strength and the risk of the approach.

5.3 Patch Generation and Edit Format

After localization, Agentless generates candidate patches. It uses a simple diff-style search/replace format rather than asking the model to rewrite entire files. This choice reduces unnecessary output and encourages small edits. It also makes patch application easier to validate. In repository-level repair, small and focused patches are often preferable because they are easier for humans to review and less likely to break unrelated behavior.

Agentless samples multiple candidate patches. This is a key design decision. Language-model outputs are stochastic, and the first patch may not be correct. Sampling creates diversity. Validation then selects among candidates. The system therefore uses the model not as a single deterministic solver but as a generator of possible repairs. This resembles generate-and-validate repair, but the candidate generator is an LLM rather than a set of hand-coded mutation operators.

The patch-generation stage also highlights a difference from SWE-agent. In SWE-agent, patch generation may occur after a long trajectory of file views and commands. In Agentless, patch generation is a stage that consumes localized context. This makes the process easier to log and compare. It also limits the model’s opportunity to gather new information during repair. The trade-off is control versus flexibility.

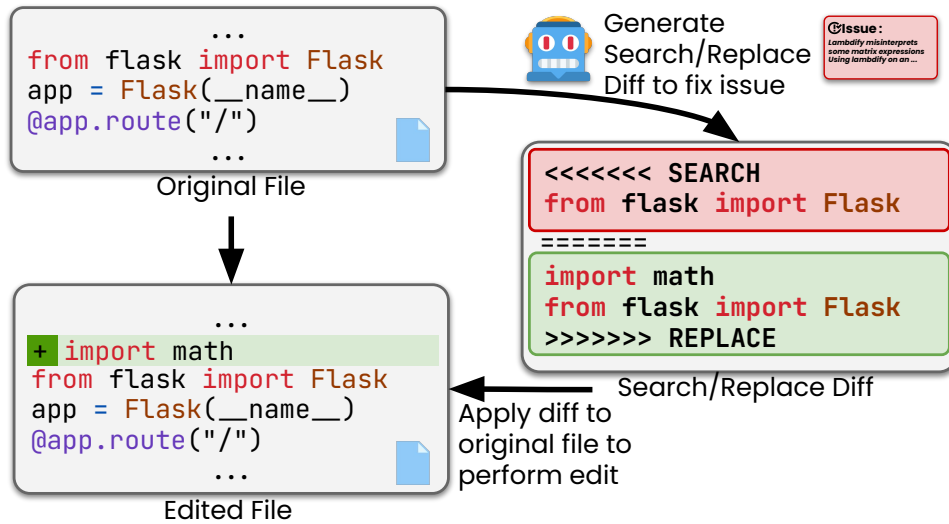


Figure 5.3: Agentless search/replace edit format. Focused diffs encourage small patches and make candidate repairs easier to apply and validate.

Table 5.2: Performance of Agentless repair setups. Sampling patches over multiple localized contexts improves the final number of resolved issues.

Repair setup	Performance	Avg. cost
Greedy location, 40 samples	88 (29.33%)	\$0.22
Multi-samples merged, 40 samples	85 (28.33%)	\$0.24
Multi-samples, 4 x 10 samples	96 (32.00%)	\$0.29

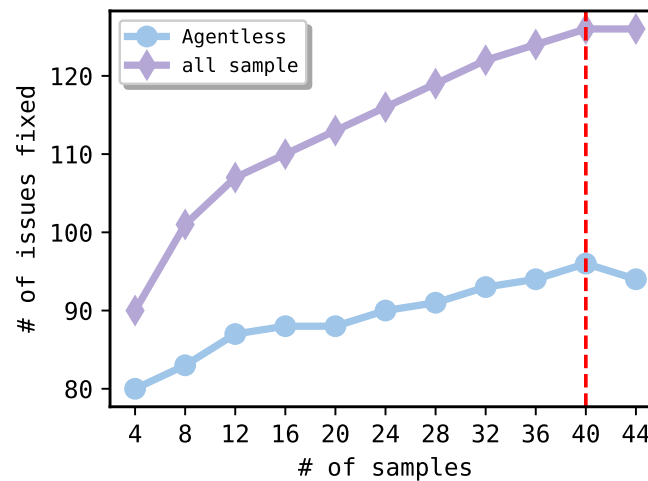


Figure 5.4: Repair performance as the number of sampled patches increases. Sampling improves coverage, but returns plateau once patch ranking becomes the bottleneck.

5.4 Reproduction Tests, Patch Validation, and Ranking

The third stage is patch validation. Agentless generates reproduction tests intended to capture the reported issue. It then uses test execution and ranking to filter candidate patches. This is an

```

from sqlfluff import lint

def test_rules_std_L060_raised() → None:
    try:
        sql = "SELECT IFNULL(NULL, 100),
                NVL(NULL,100);"
        result = lint(sql, rules=["L060"])
        assert len(result) == 2
    except:
        print("Other issues")
        return

    try:
        assert result[0]["description"] ==
"Use 'COALESCE' instead of 'IFNULL'."
        assert result[1]["description"] ==
"Use 'COALESCE' instead of 'NVL'."
        print("Issue resolved")
    except AssertionError:
        print("Issue reproduced")
        return

    return

test_rules_std_L060_raised()

```

Figure 5.5: Example Agentless reproduction test. The generated test is an executable hypothesis about the issue, later used as evidence for patch selection.

Table 5.3: Patch-validation ablation in Agentless. Generated reproduction tests provide the largest gain beyond majority voting and regression-test filtering.

Patch-selection method	Performance	Avg. cost
Majority voting	77 (25.67%)	\$0.00
+ Regression tests	81 (27.00%)	\$0.01
+ Reproduction tests	96 (32.00%)	\$0.25

important extension of simple patch generation. A patch that looks plausible may not actually reproduce or fix the issue. Validation provides an external signal.

Reproduction testing is not risk-free. A generated test may encode a wrong interpretation of the issue. It may be too narrow, selecting a patch that overfits. It may fail because of environment setup rather than behavior. Agentless handles this by sampling and filtering rather than trusting a single generated test. The pipeline uses tests as evidence, not as perfect truth.

The validation stage is where Agentless most clearly connects to automated program repair. Classical repair systems generate candidate patches and keep those that pass tests. Agentless adds LLM-based localization and patch generation, but the final logic remains test-centered. This explains why the system can be competitive without agentic autonomy. If localization is good and validation is strong, open-ended exploration may be unnecessary for many tasks.

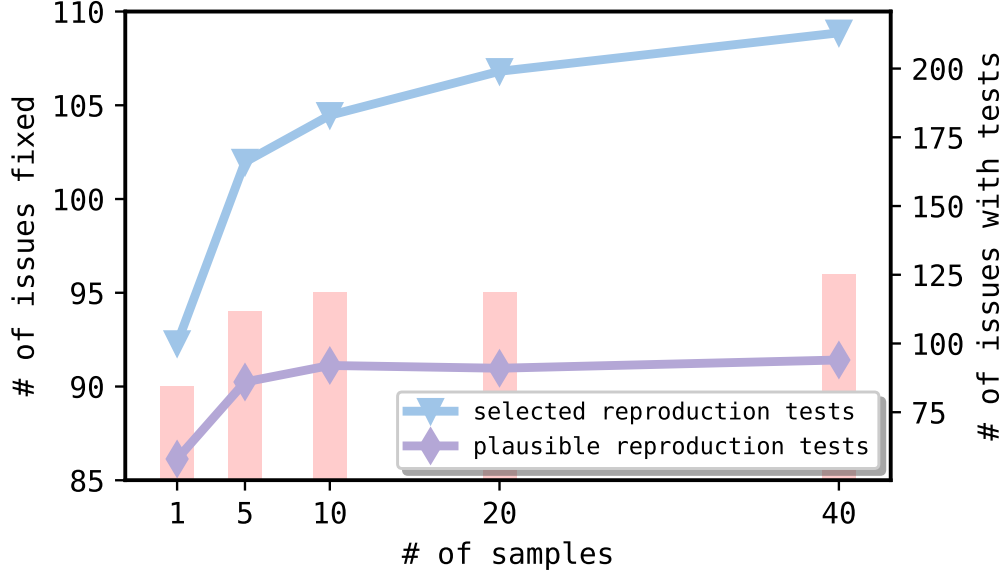


Figure 5.6: Agentless reproduction-test workflow. Candidate tests are executed on the original repository and then used with regression tests to rank candidate patches.

Table 5.4: Selected SWE-bench Lite results reported by Agentless. The table emphasizes that Agentless is competitive with agentic tools while remaining pipeline-based and relatively cheap.

Tool	Type	Resolved	Avg. cost
CodeStory Aide	closed-source agent	129 (43.00%)	–
AutoCodeRover-v2	open-source agent	92 (30.67%)	–
SWE-agent with Claude Sonnet	open-source agent	69 (23.00%)	\$1.62
RAG with Claude Opus	non-interactive baseline	13 (4.33%)	\$0.25
Agentless with GPT-4o	open-source pipeline	96 (32.00%)	\$0.70

5.5 Evaluation, Benchmark Audit, and SWE-bench Lite-S

Agentless evaluates on SWE-bench Lite and reports strong performance among open-source approaches. The result is notable because the system is intentionally simpler than many agent frameworks. It shows that structured decomposition can be a powerful baseline. The lesson is not that agents are obsolete, but that any claim about agents must compare against strong non-agentic pipelines.

The paper also introduces SWE-bench Lite-S after auditing SWE-bench Lite. This is one of its most important contributions. The audit categorizes problems and removes problematic issues to create a subset intended for more rigorous evaluation. This work shows that benchmark curation is not finished after the original dataset is released. Solvers can reveal flaws or ambiguities in benchmarks, and benchmark audits can improve scientific comparison.

The Lite-S contribution also connects back to SWE-bench’s validity risks. Real-world benchmarks are valuable because they contain authentic complexity, but authenticity can include

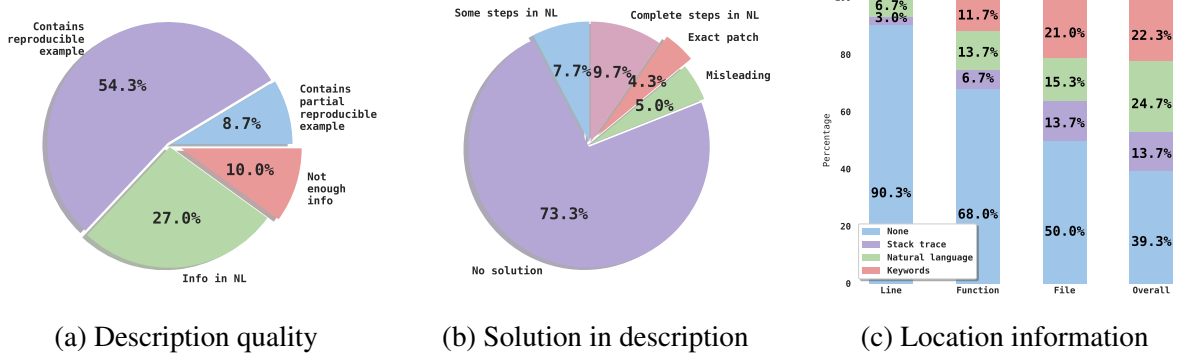


Figure 5.7: Agentless audit of SWE-bench Lite. The categories reveal that benchmark items differ substantially in information quality, patch hints, and localization clues.

Table 5.5: Selected results on SWE-bench Lite-S, the filtered benchmark subset proposed by Agentless. Filtering removes tasks with exact patches, misleading solutions, or insufficient information.

Tool	Lite resolved	Lite-S resolved	Lite-S rank
CodeStory Aide	129 (43.00%)	114 (45.78%)	1
AutoCodeRover-v2	92 (30.67%)	79 (31.73%)	11
SWE-agent with Claude Sonnet	69 (23.00%)	58 (23.29%)	22
RAG with Claude Opus	13 (4.33%)	10 (4.02%)	30
Agentless	96 (32.00%)	84 (33.73%)	9

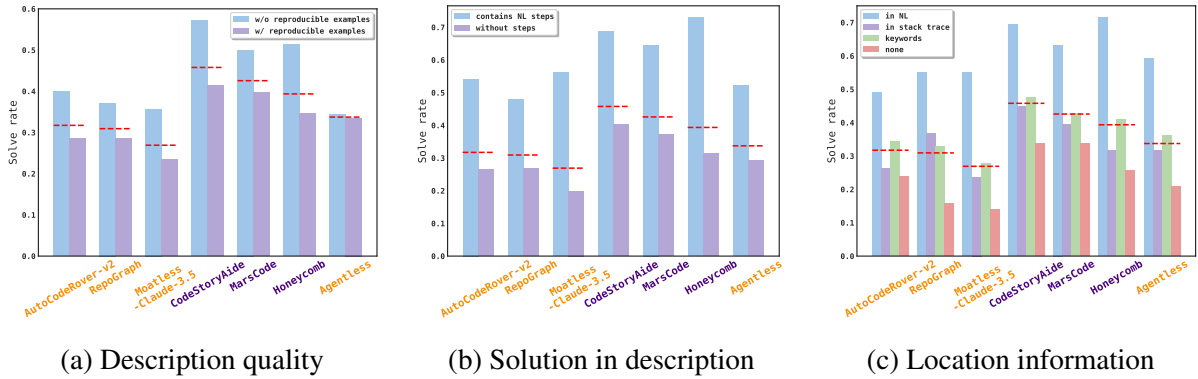


Figure 5.8: Solve-rate breakdowns on SWE-bench Lite-S. The filtered analysis makes task difficulty more visible than a single leaderboard score.

ambiguous issues, brittle tests, or tasks that do not cleanly measure the intended ability. Agentless demonstrates that system builders should not only optimize against benchmarks; they should also inspect them.

5.6 Strengths, Limitations, and Relationship to Agents

Agentless has three major strengths. First, its stages are interpretable. A human can inspect the localized files, proposed edit locations, candidate patches, generated tests, and validation

decisions. Second, its control flow is bounded. This makes cost and behavior easier to predict than in an open-ended agent loop. Third, it provides a strong baseline that prevents the field from over-crediting autonomy.

The limitations follow from the same design. A pipeline can fail early if localization is wrong. It may not recover from unexpected observations because it does not have a flexible action loop. It may struggle with issues that require exploratory debugging, environment investigation, or multi-step reasoning across dynamic behavior. It also depends on the quality of generated reproduction tests and the reliability of validation.

The relation to SWE-agent is therefore not adversarial in a simple sense. SWE-agent and Agentless explore different points in the design space. SWE-agent prioritizes interactive adaptability. Agentless prioritizes decomposition and auditability. A practical system could use Agentless-style localization and patch generation first, then escalate difficult or low-confidence cases to a SWE-agent-style interactive agent.

5.7 Agentless as a Benchmark-Aware Repair System

Agentless is benchmark-aware in two senses. It is designed for the structure of SWE-bench-style tasks, and it studies the benchmark itself. This dual role is important. Many systems treat a benchmark as a fixed scoreboard. Agentless treats it as an object to be audited, filtered, and better understood. This makes the paper a contribution to both method design and evaluation methodology.

The paper also clarifies what “Agentless” means. It does not mean no language model, no reasoning, or no tool use. It means no open-ended autonomous control loop in which the model repeatedly decides what to do next. The LLM still performs complex subtasks, but those subtasks are embedded in a system-designed pipeline. This distinction is central to the comparative review. The real design question is not whether to use an LLM, but how much control to delegate to it.

A deeper reading of Agentless also shows that simplicity is a design constraint, not a lack of sophistication. The system uses LLMs at several points, but each use is bounded by a clear input-output contract. A localization prompt should return candidate locations. A patch-generation prompt should return edits. A reproduction-test prompt should return executable tests. A validation step should rank candidates. These contracts make the system easier to debug. If performance drops, the researcher can inspect which contract failed.

The Lite-S audit is especially significant because it turns solver development back onto benchmark design. A benchmark subset may contain tasks that are ambiguous, tasks whose tests do not match the issue, or tasks whose solution depends on non-obvious project decisions. Removing or categorizing such tasks is not cherry-picking if the goal is to measure a specific capability more cleanly. Instead, it is a form of benchmark maintenance. As models improve,

benchmarks need maintenance for the same reason software does: hidden assumptions become visible only after use.

Agentless also changes how one should think about reproduction tests. In traditional issue resolution, a developer often first writes or identifies a failing test that reproduces the bug. This test is not merely a grading tool; it is a thinking tool. It clarifies the expected behavior. Agentless automates part of that workflow. The generated reproduction test helps choose among candidate patches, but it also provides an artifact that a human could inspect. If the test is wrong, the human can see the misunderstanding. If it is right, it can become part of the final regression suite.

The pipeline’s use of multiple candidates is another important systems idea. A single LLM output is fragile. Sampling many outputs and ranking them converts stochastic generation into a search process. The search space is not arbitrary; it is constrained by localized context and edit format. This is why Agentless connects to both LLM prompting and classical repair. The LLM provides semantic flexibility, while the pipeline provides search discipline. Future systems can vary the number of candidates, localization breadth, and validation strictness to tune cost and accuracy.

5.8 Summary

Agentless completes the three-paper sequence by showing that repository-level repair does not always require an open-ended agent. Its staged design turns localization, repair, test generation, and patch selection into inspectable contracts. The system is not less model-driven; rather, it gives the model narrower responsibilities and gives the surrounding pipeline more control. This contrast with SWE-agent motivates the comparative synthesis in Chapter 6.

Chapter 6

Comparative Review: Benchmark Definition, Agentic Interaction, and Structured Pipelines

6.1 A Shared Framework for the Three Papers

The three papers are best compared as a sequence of responsibilities. SWE-bench defines the task and evaluation. SWE-agent defines an agentic interaction strategy for solving that task. Agentless defines a structured pipeline strategy for solving the same family of tasks. This shared framework avoids a common misunderstanding. SWE-bench is not a solver competing with SWE-agent or Agentless. It is the problem setting that makes their comparison meaningful.

The framework also separates three questions that are often conflated. First, what is the task? This is the SWE-bench question. Second, what information and actions should the model have access to? This is the SWE-agent question. Third, how should the repair process be controlled and validated? This is the Agentless question. Every future system must answer all three, even if it does so implicitly.

This separation helps explain why final resolve rate is not enough. Suppose one system solves more tasks but uses far more model calls. Suppose another is cheaper but fails on tasks requiring interaction. Suppose a third relies on visible tests that another system cannot use. A single leaderboard number cannot capture these differences. The comparative framework therefore emphasizes task definition, information flow, control structure, validation, and cost.

6.2 Benchmark Definition and Evaluation Validity

SWE-bench’s main contribution is benchmark definition. It creates a task that is realistic enough to stress modern models and structured enough to evaluate automatically. SWE-agent and Agentless inherit this definition. Their contributions are meaningful because they operate under

Table 6.1: Compact comparison of the three focal papers. The table summarizes the role each paper plays in the thesis’s benchmark-agent-pipeline narrative.

Paper	Primary role	Control structure	Main evidence source	Main inspectability artifact
SWE-bench	Defines the task and grading target	Benchmark harness applies patches and runs tests	Historical issues, pull requests, repository snapshots, and tests	Task instances, reference patches, evaluation logs
SWE-agent	Studies agentic interaction for the task	LLM chooses actions through an ACI	Interactive file views, search results, command output, and tests	Agent trajectory and final repository diff
Agentless	Studies staged repair for the same task family	System fixes the pipeline stages; LLM fills bounded contracts	Localized context, sampled patches, generated reproduction tests, and regression tests	Ranked files, edit locations, candidate patches, and validation results

a shared evaluation target. Without SWE-bench, claims about agentic or pipeline repair would be harder to compare.

However, the comparison also shows that benchmarks are not neutral background objects. The benchmark shapes system design. SWE-bench rewards systems that can localize relevant files, generate patches, and pass tests. It does not directly reward maintainability, security, or human preference except insofar as they affect tests. Therefore, solver papers must be careful about what their scores mean. Agentless’s Lite-S audit is especially valuable because it treats benchmark validity as part of the research problem rather than a fixed assumption.

A strong evaluation ecosystem should include multiple views of validity. Construct validity asks whether passing tests measures issue resolution. External validity asks whether Python GitHub issues generalize to other languages and organizations. Internal validity asks whether environment setup, model contamination, or prompt details explain results. Reproducibility asks whether independent researchers can rerun the system. The three papers collectively move the field toward this richer view, but they also show that more reporting is needed.

6.3 Agent-Based Interaction Versus Pipeline-Based Control

The central contrast between SWE-agent and Agentless is control. SWE-agent delegates more control to the model. The model decides which files to inspect, which commands to run, when to edit, and when to submit. Agentless delegates less control. The system designer fixes the sequence of localization, repair, reproduction testing, and validation. The model performs subtasks but does not freely steer the whole process.

Agent-based interaction is powerful when the search space is uncertain. If the issue report is vague, the relevant file is hidden, or a test failure requires investigation, an agent can gather new evidence. It can adapt to observations. It can recover from some wrong hypotheses. This flexibility resembles human debugging. The cost is that behavior becomes less predictable. The agent may wander, repeat actions, or produce long trajectories that are hard to audit.

Pipeline-based control is powerful when the task can be decomposed. If localization can identify plausible edit sites and validation can rank candidate patches, then the system may not need open-ended exploration. The pipeline is easier to inspect and reproduce. It also creates natural intermediate artifacts. The cost is brittleness. If a stage makes a wrong decision, later stages may not recover. The system may lack the creativity to investigate unexpected behavior.

The best interpretation is not that one paradigm dominates. Instead, agentic interaction and structured pipelines are tools for different uncertainty regimes. High uncertainty favors agents. Low to moderate uncertainty favors pipelines. A hybrid system could use confidence signals to choose between them.

6.4 Localization, Context, and Information Flow

Localization is the hidden bottleneck across all three papers. SWE-bench exposes the bottleneck through BM25 and oracle retrieval comparisons. SWE-agent addresses it through interactive search and file viewing. Agentless addresses it through hierarchical localization. These are three different information-flow designs.

In SWE-bench baselines, information flow is mostly front-loaded. Retrieval selects context before the model generates a patch. This is simple but fragile. In SWE-agent, information flow is iterative. The model observes, searches, edits, and observes again. This is flexible but expensive. In Agentless, information flow is staged. The system first builds a compressed view of the repository, then narrows to files and symbols, then generates patches. This is efficient but depends on the quality of each stage.

The comparison suggests that future systems should report localization metrics separately from final repair metrics. A final failed patch does not reveal whether the system looked at the wrong file or generated the wrong edit. SWE-bench’s oracle analyses, SWE-agent’s action traces, and Agentless’s localization ablations all point toward the same need: diagnose the pipeline of reasoning, not only the final output.

6.5 Validation, Tests, and Patch Plausibility

All three papers rely on tests, but they use them differently. SWE-bench uses hidden tests for evaluation. SWE-agent may run commands and tests during interaction, depending on the

trajectory. Agentless explicitly generates reproduction tests and uses validation to filter patches. These differences affect what kind of ability is being measured.

A system that passes hidden tests without seeing any test feedback demonstrates strong one-shot repair. A system that passes after running tests demonstrates interactive debugging ability. A system that generates reproduction tests demonstrates the ability to operationalize issue descriptions. These are related but distinct capabilities. Future leaderboards should report what test information was available to the solver.

Patch plausibility is another concern. Passing tests does not guarantee a maintainable fix. Classical repair work has shown that generated patches can overfit tests or exploit weaknesses in the test suite [36, 41]. LLMs may produce more readable patches than older mutation-based systems, but they can still overfit or introduce subtle errors. Therefore, a mature evaluation should combine tests with static analysis, human review, security checks, and patch minimality criteria.

6.6 Cost, Reproducibility, and Deployment

Cost is a decisive difference between agents and pipelines. SWE-agent’s interaction loop can improve performance but increases API cost and runtime. Agentless samples multiple patches and tests, but its control flow is more bounded. The right choice depends on the deployment scenario. For a high-priority production bug, an expensive agent may be acceptable. For continuous triage across thousands of issues, a cheaper pipeline may be preferable.

Reproducibility also differs. A pipeline with fixed stages is easier to rerun and analyze. An agent trajectory depends on model sampling, prompts, environment outputs, and budget limits. This does not mean agent results are invalid, but it means they require detailed logging. Reproducible agent research should publish prompts, tool definitions, model versions, action traces, cost limits, and environment configurations.

Deployment raises safety concerns. An agent that can run commands may execute untrusted code, access secrets, or modify files outside the intended scope if not sandboxed. A pipeline reduces open-ended command execution but still applies patches and runs tests. Both approaches require containerization, file-system isolation, network controls, and careful permission boundaries. These are operating-systems concerns, not merely machine-learning details.

6.7 Toward Hybrid Automated Software Engineering

The comparative review points toward hybrid systems. A practical design would begin with a SWE-bench-style task definition: an issue, a repository snapshot, and an executable validation harness. It would first run an Agentless-style pipeline: hierarchical localization, focused patch generation, reproduction testing, and candidate ranking. If the pipeline is confident and validation

succeeds, the system can stop. If localization confidence is low, generated tests conflict, or patches fail, the system can escalate to a SWE-agent-style interactive agent.

Such a hybrid design would combine efficiency and flexibility. The pipeline handles common cases cheaply. The agent handles uncertain cases adaptively. Both paths should produce artifacts for human review: selected files, reasoning summaries, candidate patches, tests run, validation results, and final confidence. The system should also record why escalation occurred and how much cost was spent.

The hybrid direction also changes research evaluation. Instead of asking whether agents or pipelines are universally better, researchers should ask which tasks require which control strategy. Benchmarks could include metadata about localization difficulty, test availability, dependency complexity, and need for interactive debugging. This would make progress more diagnostic and more useful for deployment.

A useful hybrid system should also expose confidence at each boundary. After localization, it should report whether the relevant files were found by lexical evidence, model reasoning, previous failing tests, or repository structure. After patch generation, it should report whether candidates are small, whether they modify public APIs, whether they add tests, and whether multiple candidates converge on the same edit. After validation, it should report which tests were run, which failed, which were generated, and whether the generated reproduction test is consistent with the original issue. These reports would make the final patch easier for a developer to review. They would also make benchmark results more scientific, because researchers could compare intermediate behavior rather than only final scores.

The hybrid system should be conservative about escalation. Escalating every task to an agent wastes cost and increases risk. Never escalating makes the system brittle. A decision rule could use localization entropy, number of candidate files, disagreement among sampled patches, validation conflicts, and failure traces. If the pipeline identifies one dominant file and several candidate patches agree on a small edit, the system can remain pipeline-based. If candidates disagree widely, tests fail for unexpected reasons, or the issue mentions dynamic behavior that cannot be inferred from static context, the system can start an agent. The agent should then receive the pipeline artifacts as context rather than beginning from scratch.

This hybrid view also changes how one should design benchmarks. A benchmark instance could include not only final pass/fail evaluation but also diagnostic annotations: whether the fix is single-file or multi-file, whether it requires adding tests, whether the issue text names the relevant component, whether the original pull request changed public behavior, and whether reproduction requires external dependencies. Such annotations would allow researchers to ask more precise questions. Do agents help mostly on multi-file tasks? Do pipelines dominate when issue text names a function? Does reproduction-test generation help more on regression bugs than on feature requests? These questions are more useful than a single global leaderboard.

There is also a human workflow implication. A developer may not want a fully autonomous system to merge a patch. They may want a system that returns a ranked set of suspected files, a minimal patch, a generated test, and a short explanation of evidence. In that workflow, Agentless-style artifacts are directly useful. If the developer asks the assistant to investigate further, SWE-agent-style interaction becomes useful. The human can then guide or constrain the agent. This suggests that future systems should be designed not only for autonomous benchmark solving but also for mixed-initiative development.

The operating-systems implications remain central in the hybrid setting. Both the pipeline and the agent need process isolation. Test execution should run in containers or otherwise restricted environments. Network access should be controlled. File-system writes should be limited to the repository workspace. Secrets should be unavailable. Logs should record commands and patch operations. These requirements are not optional deployment details. They shape what kinds of autonomy are safe. A system that cannot isolate execution should prefer pipeline stages with limited actions; a system with strong sandboxing can safely allow more exploratory agent behavior.

The comparison can also be summarized as five design axes: autonomy, evidence, granularity, validation, and human inspectability. SWE-bench fixes the evaluation target on these axes. SWE-agent moves toward high autonomy, interactive evidence, and full action traces. Agentless moves toward staged autonomy, generated-test evidence, and compact intermediate artifacts. This axis-based view makes future systems easier to design deliberately rather than by copying one template.

This axis-based view also explains why benchmark leaderboards are useful but incomplete. A leaderboard number does not say whether a system is cheap, safe, reproducible, or easy to review. A responsible evaluation should therefore include secondary metrics such as cost per resolved issue, patch size, number of files modified, number of tests run, generated-test quality, localization accuracy, and human review time.

For an operating-systems course, the execution model is especially relevant. A benchmark harness is effectively an automated workload manager. It checks out repositories, applies patches, runs commands, captures outputs, enforces timeouts, and isolates failures. An agent framework is an even more dynamic workload manager because the commands are chosen by a model. This creates familiar OS questions: what process privileges should the agent have, what resources should be limited, how should file writes be tracked, how should subprocesses be killed, and how should logs be preserved? The LLM is new, but the need for isolation and resource control is classical.

The human-review dimension should not be treated as secondary. In real development, an automatically generated patch is rarely useful without an explanation of why it is safe. SWE-bench evaluates correctness through tests, but a maintainer evaluates a patch through code review. SWE-agent can provide a trace of evidence, but the trace may be verbose. Agentless can provide

cleaner intermediate artifacts, but it may omit exploratory context. A hybrid system should therefore generate a concise review packet: issue summary, suspected root cause, files inspected, patch rationale, tests run, generated tests, known risks, and rollback instructions. Such a packet would make autonomous repair more compatible with existing engineering practice.

6.8 Summary

The comparative chapter reframes the three papers as complementary answers to one systems problem. SWE-bench establishes the workload, SWE-agent expands the model’s action space, and Agentless constrains the workflow into auditable stages. The central design choice is therefore not simply whether to use an LLM, but where to place control, evidence, validation, and human review within the surrounding system.

Chapter 7

Conclusion and Future Work

7.1 Main Findings

This thesis reviewed SWE-bench, SWE-agent, and Agentless as a connected research chain. SWE-bench establishes the common evaluation problem: real GitHub issue resolution in real repositories under test-based grading. SWE-agent proposes an agent-based solution strategy, emphasizing that language models need interfaces designed for their interaction patterns. Agentless proposes a pipeline-based solution strategy, showing that structured localization, patch generation, and validation can compete strongly without open-ended autonomy.

The main finding is that these papers should be read together. SWE-bench defines what counts as success. SWE-agent explores how an LLM can act when given tools and feedback. Agentless tests how much of that success can be achieved through disciplined decomposition. The three papers therefore map a design space rather than three disconnected contributions.

A second finding is that repository-level repair is fundamentally a systems problem. Model quality matters, but so do retrieval, interface design, edit mechanisms, tests, cost, environment isolation, and logging. Improvements in one component can be hidden by failures in another. For example, a strong model may fail if localization is poor. A good patch may fail if the environment is brittle. A high resolve rate may be less useful if cost is prohibitive.

7.2 Implications for LLM4Code Research

The review suggests several implications. First, future papers should separate localization performance from patch-generation performance. Oracle retrieval, localization ablations, and trace analysis are useful because they identify where systems fail. Second, papers should report cost and test visibility. A result is hard to interpret without knowing how many model calls were used and what feedback was available. Third, benchmark audits should become standard. Real-world benchmarks are valuable but not perfect.

Fourth, interface design deserves more attention. SWE-agent shows that the same model can behave differently depending on the action space and feedback format. This insight extends beyond SWE-bench. Any tool-using LLM system needs an interface that matches model capabilities. Fifth, structured baselines should remain strong. Agentless demonstrates that pipelines can be both simple and competitive. Agent papers should compare against such baselines to show that autonomy itself adds value.

7.3 Future Work

Future work should expand benchmark diversity. SWE-bench focuses on Python repositories. Real software engineering includes Java, C++, Rust, JavaScript, Go, distributed systems, mobile apps, and infrastructure code. Each domain has different build systems, dependency patterns, and testing practices. A broader benchmark ecosystem would reveal which methods generalize.

Future work should also improve validation. Hidden tests are necessary but not sufficient. Systems should integrate static analysis, security checks, style checks, mutation testing, and human review. LLM-generated reproduction tests are promising, but they need reliability measures. A generated test should be treated as evidence with confidence, not as ground truth.

Another direction is human-AI collaboration. The three focal papers emphasize autonomous or mostly autonomous solving. In practice, developers may want assistants that propose patches, explain evidence, and ask clarifying questions. Evaluation should therefore include human usefulness, review time, and trust. A patch that does not fully solve an issue may still be valuable if it localizes the bug or produces a good reproduction test.

Finally, future systems should be designed with safety and reproducibility from the beginning. Agents and pipelines should run in sandboxes, record complete traces, expose cost, and preserve intermediate artifacts. These requirements are essential if automated software engineering is to move from benchmark demonstrations to reliable engineering tools.

7.4 Closing Reflection

SWE-bench, SWE-agent, and Agentless together show that LLM4Code has moved beyond isolated code generation. The field now studies benchmarks, environments, agents, pipelines, tests, and human review. This is a healthy development. Software engineering is not a single prediction problem. It is a process of understanding, changing, and validating complex systems. The most successful future methods will likely be those that combine realistic benchmarks, carefully designed interfaces, structured repair pipelines, and transparent validation.

Bibliography

- [1] Rui Abreu, Peter Zoetewij, and Arjan J. C. van Gemund. “On the Accuracy of Spectrum-Based Fault Localization.” *Testing: Academic and Industrial Conference Practice and Research Techniques*, 2007.
- [2] Miltiadis Allamanis, Earl T. Barr, Premkumar Devanbu, and Charles Sutton. “A Survey of Machine Learning for Big Code and Naturalness.” *ACM Computing Surveys*, 2018.
- [3] Jacob Austin et al. “Program Synthesis with Large Language Models.” *arXiv preprint arXiv:2108.07732*, 2021.
- [4] Earl T. Barr, Yuriy Brun, Premkumar Devanbu, Mark Harman, and Federica Sarro. “The Plastic Surgery Hypothesis.” *International Symposium on Foundations of Software Engineering*, 2014.
- [5] Federico Cassano et al. “MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation.” *IEEE Transactions on Software Engineering*, 2023.
- [6] Mark Chen et al. “Evaluating Large Language Models Trained on Code.” *arXiv preprint arXiv:2107.03374*, 2021.
- [7] Xiang Deng et al. “Mind2Web: Towards a Generalist Agent for the Web.” *Advances in Neural Information Processing Systems*, 2023.
- [8] Gordon Fraser and Andrea Arcuri. “EvoSuite: Automatic Test Suite Generation for Object-Oriented Software.” *Joint Meeting on Foundations of Software Engineering*, 2011.
- [9] Daniel Fried et al. “InCoder: A Generative Model for Code Infilling and Synthesis.” *International Conference on Learning Representations*, 2023.
- [10] Timnit Gebru et al. “Datasheets for Datasets.” *Communications of the ACM*, 2021.
- [11] Ali Ghanbari, Samuel Benton, and Lingming Zhang. “Practical Program Repair via Byte-code Mutation.” *International Symposium on Software Testing and Analysis*, 2019.
- [12] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. “Measuring

- Coding Challenge Competence with APPS.” Advances in Neural Information Processing Systems, 2021.
- [13] Sirui Hong et al. “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework.” International Conference on Learning Representations, 2024.
 - [14] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. “CodeSearchNet Challenge: Evaluating the State of Semantic Code Search.” arXiv preprint arXiv:1909.09436, 2019.
 - [15] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” International Conference on Learning Representations, 2024.
 - [16] James A. Jones and Mary Jean Harrold. “Empirical Evaluation of the Tarantula Automatic Fault-Localization Technique.” International Conference on Automated Software Engineering, 2005.
 - [17] Rene Just, Darioush Jalali, and Michael D. Ernst. “Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs.” International Symposium on Software Testing and Analysis, 2014.
 - [18] Sayash Kapoor and Arvind Narayanan. “Leakage and the Reproducibility Crisis in Machine-Learning-Based Science.” Patterns, 2023.
 - [19] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. “Automatic Patch Generation Learned from Human-Written Patches.” International Conference on Software Engineering, 2013.
 - [20] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. “GenProg: A Generic Method for Automatic Software Repair.” IEEE Transactions on Software Engineering, 2012.
 - [21] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. “Automated Program Repair.” Communications of the ACM, 2019.
 - [22] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. “CODAMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models.” International Conference on Software Engineering, 2023.
 - [23] Raymond Li et al. “StarCoder: May the Source Be With You!” Transactions on Machine Learning Research, 2023.
 - [24] Yujia Li et al. “Competition-Level Code Generation with AlphaCode.” Science, 2022.

- [25] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. “Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation.” *Advances in Neural Information Processing Systems*, 2023.
- [26] Fan Long and Martin Rinard. “Automatic Patch Generation by Learning Correct Code.” *Principles of Programming Languages*, 2016.
- [27] Shuai Lu et al. “CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation.” *Advances in Neural Information Processing Systems Datasets and Benchmarks*, 2021.
- [28] Valentin J. M. Manes et al. “The Art, Science, and Engineering of Fuzzing: A Survey.” *IEEE Transactions on Software Engineering*, 2019.
- [29] Fernando Martinez-Plumed et al. “Research Community Dynamics Behind Popular AI Benchmarks.” *Nature Machine Intelligence*, 2021.
- [30] Margaret Mitchell et al. “Model Cards for Model Reporting.” *Conference on Fairness, Accountability, and Transparency*, 2019.
- [31] Martin Monperrus. “Automatic Software Repair: A Bibliography.” *ACM Computing Surveys*, 2018.
- [32] Lee Naish, Hua Jie Lee, and Kotagiri Ramamohanarao. “A Model for Spectra-Based Software Diagnosis.” *ACM Transactions on Software Engineering and Methodology*, 2011.
- [33] Erik Nijkamp et al. “CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis.” *International Conference on Learning Representations*, 2023.
- [34] Carlos Pacheco and Michael D. Ernst. “Randoop: Feedback-Directed Random Testing for Java.” *Object-Oriented Programming, Systems, Languages, and Applications Companion*, 2007.
- [35] Joelle Pineau et al. “Improving Reproducibility in Machine Learning Research.” *Journal of Machine Learning Research*, 2021.
- [36] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. “An Analysis of Patch Plausibility and Correctness for Generate-and-Validate Patch Generation Systems.” *International Symposium on Software Testing and Analysis*, 2015.
- [37] Chen Qian et al. “Communicative Agents for Software Development.” *arXiv preprint arXiv:2307.07924*, 2023.

- [38] Baishakhi Ray, Vincent Hellendoorn, Saheel Godhane, Zhaopeng Tu, Alberto Bacchelli, and Premkumar Devanbu. “On the Naturalness of Buggy Code.” International Conference on Software Engineering, 2016.
- [39] Timo Schick et al. “Toolformer: Language Models Can Teach Themselves to Use Tools.” Advances in Neural Information Processing Systems, 2023.
- [40] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. “Reflexion: Language Agents with Verbal Reinforcement Learning.” Advances in Neural Information Processing Systems, 2023.
- [41] Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. “Is the Cure Worse Than the Disease? Overfitting in Automated Program Repair.” Joint Meeting on Foundations of Software Engineering, 2015.
- [42] Aarohi Srivastava et al. “Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models.” Transactions on Machine Learning Research, 2023.
- [43] Michele Tufano et al. “An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation.” ACM Transactions on Software Engineering and Methodology, 2019.
- [44] Guanzhi Wang et al. “Voyager: An Open-Ended Embodied Agent with Large Language Models.” Transactions on Machine Learning Research, 2023.
- [45] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. “CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation.” Empirical Methods in Natural Language Processing, 2021.
- [46] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. “A Survey on Software Fault Localization.” IEEE Transactions on Software Engineering, 2016.
- [47] Qingyun Wu et al. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.” arXiv preprint arXiv:2308.08155, 2023.
- [48] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. “Agentless: Demystifying LLM-based Software Engineering Agents.” arXiv preprint arXiv:2407.01489, 2024.
- [49] John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. “InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback.” Advances in Neural Information Processing Systems, 2023.

- [50] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.” Advances in Neural Information Processing Systems, 2024.
- [51] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” International Conference on Learning Representations, 2023.
- [52] Michal Zalewski. “American Fuzzy Lop.” Software testing tool, 2016.
- [53] Shuyan Zhou et al. “WebArena: A Realistic Web Environment for Building Autonomous Agents.” International Conference on Learning Representations, 2024.

Appendix A

Supplementary Notes on Benchmarks and Repair Systems

A.1 Supplementary Notes on SWE-bench Variants

SWE-bench, SWE-bench Lite, and audited subsets such as SWE-bench Lite-S should be read as related but not identical evaluation settings. The full benchmark emphasizes broad repository-level realism. The Lite split makes comparison cheaper and more repeatable. Audited subsets make benchmark quality more explicit by identifying issues whose descriptions, patches, or tests may not cleanly measure the intended repair ability. These variants are complementary: broad benchmarks reveal realism, while curated subsets reveal methodological clarity.

A.2 Supplementary Notes on Model and Tool Assumptions

The three focal papers assume different relationships between the model and its environment. Static baselines assume retrieved context is enough. SWE-agent assumes the model can benefit from iterative interaction through a carefully designed ACI. AGENTLESS assumes the system can decompose the work into bounded model calls and executable validation. These assumptions affect cost, latency, safety, and reproducibility as much as they affect final accuracy.

A.3 Closing Note on Supplementary Material

The appendix reinforces the main thesis rather than adding a separate argument. Repository-level repair is best evaluated by separating task definition, control structure, evidence, and validation. That separation makes it easier to compare systems fairly and to design future hybrid tools that are both effective and reviewable.

Table A.1: Supplementary comparison of evaluation assumptions across the three focal papers.

Assumption	SWE-bench	SWE-agent	AGENTLESS
Primary object	Benchmark instance and test harness	Agent trajectory inside a repository	Staged localization-repair-validation pipeline
Context access	Retrieved or oracle files in baselines	Iterative search, file viewing, shell output, and edit feedback	Repository structure, skeletons, localized snippets, and generated tests
Cost driver	Model context length and patch attempts	Number of interaction turns and model calls	Number of localization, patch, and test samples
Main reproducibility need	Stable repositories, tests, and patch application	Logged prompts, actions, observations, budgets, and environment state	Logged stage outputs, candidate patches, tests, rankings, and fallback rules